

Ngữ nghĩa, vật thể động và đồ thị cảnh (Semantics, Dynamics and Scene Graphs)

PHỤ THUỘC C/14-bai-toan-cv-loi (detection và segmentation là gì — chương này chỉ nói về việc *dùng* chúng); F/27-dat-hoc-sau-vao-dau (khung bốn cổng); F/30-do-sau-dong-quang-va-mo-hinh-nen (dòng quang và độ sâu, hai nguồn bằng chứng hình học cho “đang động”); B/08-dich-chuyen-phan-phoi (mặt nạ hồng đúng lúc bạn cần nó nhất).

TÓM TẮT

SLAM cổ điển giả định thế giới đứng yên. Giả định đó không nằm trong dòng code nào, nhưng mọi hàm giá đều dựa vào nó. Chương này chỉ ra chỗ giả định gây, rồi hỏi ngữ nghĩa mua lại được bao nhiêu và với giá nào. Đọc xong bạn tách được ba việc mà tài liệu hay trộn làm một: vật *có thể* động, vật *đang* động, và vật *đã* động rồi lại đứng yên. Bạn cũng có một bảng xếp hạng định tính về sức nặng của các bộ nhận thức, kèm phép đo mà bạn phải tự chạy trên máy mình, để biết mô hình nào có cửa trong vòng lặp và mô hình nào chỉ dùng được ngoại tuyến. Nếu chỉ nhớ một điều: mặt nạ ngữ nghĩa là một điều chỉnh *trọng số*, không phải một lệnh *xoá* — xoá quá tay làm hỏng điều kiện của bài toán tối ưu, và hỏng điều kiện là kiểu hỏng im lặng, không có cờ ngoại lai nào báo.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mặt nạ và phân đoạn thực thể** — mặt nạ là một ảnh nhị phân cùng kích thước ảnh gốc, đánh dấu từng điểm ảnh thuộc hay không thuộc một vật; ở đây nó nói “đừng tin các phép đo rơi vào vùng này”. Phân đoạn thực thể là việc tách *từng* vật riêng lẻ thành một mặt nạ riêng kèm nhãn lớp, khác với phân đoạn ngữ nghĩa vốn gộp hai người đứng cạnh nhau làm một.
- **vật thể động** — trong chương này luôn phải nói rõ nghĩa nào trong ba nghĩa: thuộc lớp *có khả năng* tự di chuyển, *đang* thật sự di chuyển ở khung hình này, hay *đã* đổi chỗ giữa hai lần robot đi qua.
- **điểm bản đồ và phần dư tái chiếu** — điểm bản đồ là một điểm 3D mà hệ thống tin là cố định. Phần dư tái chiếu là khoảng cách pixel giữa chỗ ta chiếu nó xuống ảnh và chỗ ta thật sự quan sát nó. Nếu điểm không hề cố định, phần dư đó *không* còn chỉ nói về sai số tư thế camera.
- **ma trận thông tin** — nghịch đảo của hiệp phương sai, tức là trọng số của một ràng buộc trong bài toán tối ưu. Đặt trọng số bằng không chính là *xoá* ràng buộc; hạ trọng số là giữ nó lại nhưng bớt tin.
- **điều kiện của bài toán** — mức độ “đủ mọi hướng” của tập ràng buộc. Nếu mọi điểm còn lại nằm dồn về một phía ảnh, tư thế camera vẫn giải được nhưng có một hướng gần như không bị ràng buộc. Sai số theo hướng đó lớn mà *không ai báo*.
- **tập nhãn đóng và mở** — tập đóng nghĩa là mô hình chỉ biết đúng những lớp có trong tập huấn luyện, thường là 80 lớp của COCO. Tập mở nghĩa là mô hình nhận một chuỗi chữ bất kỳ làm tên lớp lúc chạy, qua một *prompt* — một gợi ý dạng điểm bầm, khung, hoặc câu chữ.
- **bản đồ đặc** — cách biểu diễn bề mặt bằng lưới ô lập phương hoặc lưới tam giác, thay vì bằng một đám mây điểm rời rạc. Nó tốn bộ nhớ hơn *nhiều bậc* so với bản đồ điểm thưa mà ORB-SLAM3 dựng.
- **độ trễ đuôi** — phân vị cao của phân bố độ trễ, ví dụ p99. Với vòng lặp thời gian thực, chính *đuôi* quyết định có rớt khung hay không, chứ không phải trung bình.

CÂU HỎI MỞ ĐẦU

- Q1. Một người ngồi yên trên ghế chiếm một mảng lớn liên khối của khung hình. Mặt nạ “person” của bạn xoá đúng vùng đó, và quỹ đạo tệ đi. Vì sao xoá đúng vật lại làm hỏng kết quả?
- Q2. RANSAC đã là một bộ loại ngoại lai rất tốt và bạn đã dùng nó nhiều năm. Vậy chính xác thì mặt nạ ngữ nghĩa bắt được loại ngoại lai nào mà RANSAC không bắt được?
- Q3. Vì sao mạng phân đoạn của bạn hỏng nặng nhất đúng ở những khung hình mà bạn cần nó nhất, và điều đó có phải trùng hợp không?
- Q4. Ba hệ thống cùng báo “xử lý được cảnh động”: một cái nhìn nhãn lớp, một cái nhìn phần dư hình học, một cái nhìn lịch sử bản đồ. Chúng giải ba bài toán khác nhau ở chỗ nào?
- Q5. Nếu đầu ra của hệ thống bạn chỉ là một quỹ đạo và một con số APE, thì tầng ngôn ngữ mở kiểu “tìm cho tôi cái cốc” giúp được gì — và câu trả lời trung thực nói lên điều gì về việc nên đầu tư vào đâu?

1.1 Đặt vấn đề

ORB-SLAM3 không có khái niệm “vật thể”. Mô hình thế giới của nó là một đám mây điểm 3D, mỗi điểm được giả định nằm cố định trong một hệ quy chiếu cố định [1]. Giả định đó không được viết ra ở đâu cả. Nhưng mọi hàm giá trong hệ đều dựa vào nó.

Hãy nhìn kỹ một phần dư tái chiếu. Nó là hiệu giữa chỗ ta chiếu một điểm bản đồ xuống ảnh và chỗ ta thật sự đo được điểm đó. Ta đọc phần dư ấy như sai số tư thế camera. Cách đọc đó chỉ đúng khi điểm không tự di chuyển. Nếu điểm có di chuyển, phần dư trộn hai ẩn số vào một con số. Bộ tối ưu không biết điều đó, nên nó gỡ rối theo cách duy nhất nó biết: dịch camera. Một vật động vì thế không tạo ra ngoại lai ngẫu nhiên. Nó tạo ra một sai lệch *có hướng*, và sai lệch có hướng là thứ nguy hiểm nhất với một bộ ước lượng bình phương tối thiểu.

A. Vì sao OpenLORIS khó hơn EuRoC

Bạn đã có bằng chứng cho điều này trong chính bảng kết quả của mình. EuRoC quay trong một sảnh máy bay và một phòng thí nghiệm, gần như không có người trong khung hình. OpenLORIS quay ở văn phòng, nhà ở, siêu thị và hành lang, từ độ cao của một robot dịch vụ, lặp lại nhiều lần trong nhiều tuần [2]. Chênh lệch độ khó giữa hai tập không nằm ở chuyển động của camera. Nó nằm ở nội dung cảnh, và nội dung đó phá vỡ ba giả định cùng lúc.

- **Có người đi lại** — giả định thế giới tĩnh gãy trong lúc đang chạy, ở quy mô vài giây.
- **Cảnh đổi giữa các lần đi qua** — ghế bị đẩy, hộp bị chuyển, cửa mở rồi đóng. Bản đồ cũ vẫn còn điểm ở chỗ không còn gì cả.
- **Bề mặt trắng và sàn bóng** — số đặc trưng còn lại vốn đã ít, nên bất kỳ việc xoá bớt nào cũng đắt hơn hẳn so với trên EuRoC.

Điểm thứ ba quan trọng hơn về ngoài của nó, và ta sẽ quay lại. Trên một cảnh giàu vân, xoá một phần đáng kể số điểm gần như không mất gì. Trên một hành lang trắng, đó có thể là toàn bộ khoảng cách giữa bám được và mất bám.

B. Ngữ nghĩa là kênh đưa tri thức “cái gì có thể di chuyển” vào hệ

Cần phát biểu cho hẹp lại, vì cụm “hiểu cảnh” quá rộng để làm việc được. Với một hệ đo quỹ đạo, ngữ nghĩa làm đúng một việc: nó đưa vào hệ một mẫu tri thức tiên nghiệm mà hình học không thể tự suy ra được từ hai khung hình liên tiếp — *loại vật nào trên đời này có khả năng tự di*

chuyển. Hình học chỉ nhìn thấy điểm ảnh và phần dư. Nó không biết một mảng điểm ảnh là cái ghế hay là bức tường. Ngữ nghĩa biết, vì nó đã xem hàng triệu ảnh có nhãn.

Đó là tất cả những gì tầng ngữ nghĩa cho không. Mọi thứ khác — vật này *có* đang động không, nó động lúc nào, nó sẽ còn ở đó lần sau không — vẫn phải do hình học hoặc do bộ nhớ của bản đồ trả lời.

Còn một việc thứ hai, lớn hơn nhiều nhưng phục vụ người khác. Ngữ nghĩa cho bản đồ một *từ vựng*. Một bản đồ có từ vựng dùng được bởi một bộ lập kế hoạch hoặc một con người, chứ không chỉ bởi một script tính APE. Hai việc này thường bị viết chung trong cùng một paper, và trộn chúng là nguồn nhầm lẫn lớn nhất khi đọc mảng này. Nửa sau của chương tách chúng ra.

Học chương này thì làm được gì mà trước đó không làm được? Bạn đọc một paper “semantic dynamic SLAM” và nói ngay được nó đang giải bài nào trong ba bài, nó nặng cỡ nào so với ngân sách bạn còn trống, và bảng kết quả của nó có chuyển sang robot của bạn được không. Trong phần lớn trường hợp câu trả lời là không, và biết lý do sẽ tiết kiệm cho bạn vài tháng.

TRỰC GIÁC

Một mặt nạ ngữ nghĩa là danh sách nhãn chứng mà bạn quyết định không tin, dựa trên nghề nghiệp của họ chứ không dựa trên lời khai. Loại nhãn chứng theo nghề rất rẻ và thường đúng, vì có những nghề hay nói dối thật. Nhưng giả sử tất cả người làm nghề đó tình cờ đứng ở nửa trái căn phòng. Sau khi loại họ, mọi lời khai còn lại đều đáng tin tuyệt đối — và tất cả đều nói về nửa phải. Bạn không còn cách nào dựng lại chuyện xảy ra ở nửa trái. Đó chính xác là chuyện xảy ra với điều kiện của bài toán tối ưu khi mặt nạ xoá một mảng lớn liên khối của ảnh: phần dư còn lại nhỏ và đẹp, còn tư thế thì có một hướng không ai ràng buộc.

C. Chương này chạm nỗi đau nào

Trung thực về phạm vi trước khi đi tiếp. Trong bốn nỗi đau của Phần F, chương này chạm thẳng vào **cảnh động**, một nỗi đau mà EuRoC không hề đo và vì thế dễ bị bỏ quên. Nó chạm *gián tiếp* vào **đóng vòng**, theo hai đường: một vòng sai có thể sinh ra vì hai khung hình giống nhau ở chỗ có cùng một chiếc xe đẩy chứ không phải cùng một hành lang; và đồ thị cảnh phân tầng đề xuất một cách tìm ứng viên vòng khác hẳn cách dùng bộ mô tả ảnh toàn cục.

Nó gần như không chạm vào **chuyển động nhanh**, **ảnh mờ** và **trọng số IMU**. Tệ hơn thế: chuyển động nhanh và ảnh mờ làm cho chính tầng ngữ nghĩa hỏng, nên chương này *thừa hưởng* hai nỗi đau đó chứ không chữa chúng. Đó là một trong những kết luận đáng nhớ nhất ở đây, và mục cuối sẽ nói kỹ.

1.2 Công cụ nhận thức chạy được trong vòng lặp (Perception Front-Ends in the Loop)

Chương 14 đã dạy các mô hình này là gì: một giai đoạn khác hai giai đoạn ở đâu, vì sao **triệt phi cực đại** (*non-maximum suppression*) là heuristic cuối cùng, DETR bỏ nó bằng **ghép cặp Hungary** (*Hungarian matching*) thế nào, và phân đoạn hợp nhất được ba bài toán ra sao. Mục này không nhắc lại cơ chế. Nó hỏi một câu khác hẳn, câu mà một kỹ sư hệ thống thời gian thực phải hỏi trước tiên: *cái nào bỏ vừa vào ngân sách của tôi, và bỏ vào chỗ nào*.

A. Ngân sách thật trên Orin, trước khi bàn tới mô hình nào

Bắt đầu từ ràng buộc chứ đừng bắt đầu từ bảng mAP. Ngân sách của một vòng theo vết là chu kỳ khung hình, không hơn. Trong ngân sách đó, một hệ stereo-inertial đã tiêu phần lớn cho trích xuất đặc trưng, ghép cặp và tối ưu tư thế. Phần còn trống cho một mạng nhận thức chỉ là phần thừa ra sau ba việc ấy, và nó hẹp hơn nhiều so với trực giác. Bao nhiêu thì *chỉ máy của bạn trả lời được*, và phép đo thì rẻ: chạy hệ SLAM một mình trên phần cứng thật, ghi phân vị cao của chu kỳ luồng theo vết, rồi lấy chu kỳ khung trừ đi con số đó. Phần dư ra mới là ngân sách thật, và nó thường nhỏ tới mức bất ngờ.

Ba sự thật về phần cứng quyết định thiết kế, và cả ba đều hay bị bỏ qua khi đọc bảng FPS trong paper.

- **GPU là tài nguyên dùng chung.** Trên Jetson Orin không có GPU rời thứ hai. Nếu bạn đã chuyển trích xuất ORB lên GPU, mạng nhận thức và bộ trích xuất tranh nhau cùng các nhân tính toán và cùng băng thông bộ nhớ. “Chạy song song” trên giấy trở thành chạy xen kẽ trên máy, và độ trễ của cả hai đều tăng.
- **Bộ tăng tốc học sâu chuyên dụng là đòn bẩy song song thật.** Dòng AGX Orin có hai khối tăng tốc học sâu tách rời khỏi GPU. Đưa mạng nhận thức xuống đó là cách duy nhất để nó thật sự chạy song song với luồng theo vết. Cái giá là tập phép toán được hỗ trợ hẹp hơn nhiều, nên phần lớn mạng phải sửa mới chạy nổi, và một số tầng vẫn rơi ngược về GPU.
- **Đuôi mới giết bạn, không phải trung bình.** Một mạng có trung bình nằm gọn trong ngân sách nhưng có phân vị 99 vượt hẳn ngân sách sẽ làm rớt khung đúng lúc cảnh phức tạp. Với các detector còn dùng triết phi cực đại, thời gian hậu xử lý tỉ lệ với số khung sống sót, nên đuôi độ trễ dày lên đúng ở cảnh đông người — tức là đúng lúc bạn cần mặt nạ.

Từ ba điều đó suy ra một kết luận kiến trúc, và nó quan trọng hơn việc chọn mô hình nào. **Tầng nhận thức không nên nằm trong đường tới hạn của luồng theo vết.** Cấu hình thực dụng là chạy mạng trên một luồng riêng, ở tần số thấp hơn, chỉ trên khung hình chính, rồi truyền mặt nạ sang các khung xen giữa bằng một cơ chế rẻ. Hình 1.1 vẽ cả hai cách nổi, và cho thấy vì sao cách đồng bộ hỏng.



Hình 1.1. Hai cách nối tầng nhận thức vào vòng lặp. Cách đồng bộ cộng thẳng độ trễ mạng vào ngân sách mỗi khung, và đuôi độ trễ dày lên đúng ở cảnh đông người. Cách bất đồng bộ giữ nhịp cho luồng theo vết nhưng đổi lấy một mặt nạ luôn lạc hậu và khung — sai lệch đó tỉ lệ với tốc độ của vật, nên nó lớn nhất đúng ở trường hợp mặt nạ đáng giá nhất.

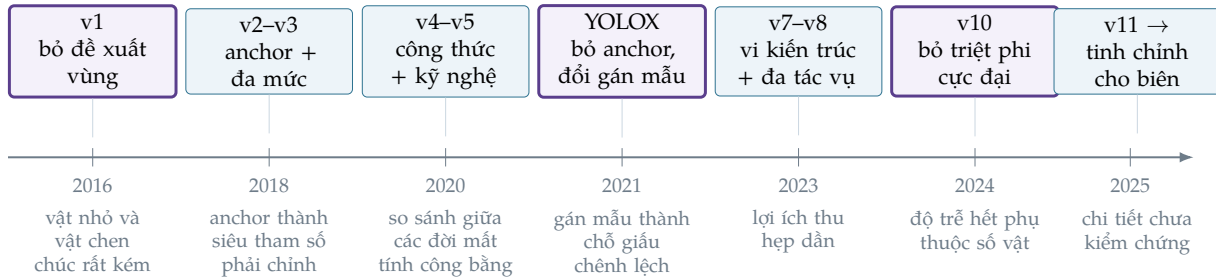
B. Dòng YOLO: cái gì thật sự đổi qua các đời

Mười một phiên bản trong tám năm tạo ấn tượng về một chuỗi đột phá liên tiếp. Đọc kỹ thì không phải vậy. Phần lớn chênh lệch giữa các đời đến từ ba nguồn ít được viết trong tiêu đề: quy tắc gán mẫu, công thức huấn luyện, và kỹ nghệ triển khai. Chỉ có vài bước thật sự đổi cấu trúc bài toán. Với một kỹ sư SLAM, chỉ hai bước trong số đó đáng quan tâm.

- **v1 (2016)** — bỏ hẳn giai đoạn đề xuất vùng, hồi quy thẳng trên một lưới [3]. Đây là lần đổi phát biểu thật sự đầu tiên, và nó mua lấy tốc độ bằng cách hy sinh vật nhỏ và vật chen chúc.
- **v2, v3 (2017–2018)** — *anchor* lấy từ phân cụm kích thước hộp, rồi dự đoán trên ba mức độ phân giải. Bước thứ hai chứa đúng điểm yếu vật nhỏ của v1.
- **v4 (2020)** — một tập hợp thủ thuật huấn luyện và một cái *neck* tốt hơn, gói lại thành “bag of freebies” [4]. Đóng góp thật nằm ở khảo sát có hệ thống xem thủ thuật nào cộng dồn được, chứ không ở một ý tưởng mới.
- **v5 (2020)** — không có bài báo. Cái đổi là kỹ nghệ: đóng gói, xuất mô hình, huấn luyện lại dễ. Với người triển khai, đây là phiên bản đổi nhiều nhất trong đời thực, và đó là một nhận xét về ngành chứ không phải về thuật toán.
- **YOLOX (2021) và v8 (2023)** — bỏ *anchor*, tách đầu phân loại khỏi đầu hồi quy, và đổi quy tắc gán mẫu [5]. Đây là bước đổi cấu trúc thứ hai, và Chương 14 đã cho thấy vì sao gán mẫu mới là chỗ giấu phần lớn chênh lệch.
- **v6, v7, v9, v11 (2022–2024)** — vi kiến trúc *backbone* và *neck*: khôi tái tham số hoá cho thân thiện với lượng tử hoá, luồng thông tin phụ lúc huấn luyện, khối chú ý nhẹ [6]. Có lợi, nhưng lợi ích thu hẹp dần qua từng đời và không đổi cách bạn nối nó vào hệ.
- **v10 (2024)** — huấn luyện với hai phép gán nhất quán để bỏ hẳn triết phi cực đại lúc suy luận [7]. Đây là bước đáng quan tâm nhất cho hệ thời gian thực, vì nó cắt đúng cái phần hậu xử lý có thời gian chạy phụ thuộc nội dung cảnh.

Dòng này vẫn tiếp tục sau v11; tới 2025–2026 đã có các bản đi tiếp theo hướng bỏ triệt phi cực đại và tối ưu cho biên. Tôi không kiểm chứng được chi tiết của chúng nên không mô tả số liệu ở đây. Hình 1.2 xếp cả dòng trên một trục và ghi rõ mỗi bước gỡ nút thắt nào.

Khung viên tím: ba bước thật sự đổi cấu trúc bài toán. Các bước còn lại đổi công thức, vi kiến trúc hoặc kỹ nghệ.



Hình 1.2. Dòng YOLO đọc theo nút thắt chứ không theo số phiên bản. Dòng dưới mỗi mốc ghi vấn đề còn lại sau bước đó. Chỉ ba mốc đổi cấu trúc: bỏ đề xuất vùng, bỏ anchor kèm đổi gán mẫu, và bỏ triệt phi cực đại. Với hệ thời gian thực, mốc cuối đáng giá nhất vì nó cắt phần hậu xử lý có thời gian chạy phụ thuộc nội dung cảnh.

C. Họ DETR: vì sao “không triệt phi cực đại” là một tính chất hệ thống

DETR đổi phát biểu thành dự đoán một tập hợp, ghép với chân trị bằng ghép cặp Hungary một-một, nên mỗi vật chỉ có đúng một dự đoán được thưởng và mô hình tự học cách không trùng lặp [8]. Chương 14 đã giải thích cơ chế đó. Điều Chương 14 chưa nói là vì sao một kỹ sư thời gian thực nên quan tâm.

Lý do không phải độ chính xác. Lý do là *hình dạng phân bố độ trễ*. Một detector có triệt phi cực đại tiêu thời gian tỉ lệ với số khung sống sót sau ngưỡng điểm. Số đó nhảy vọt ở cảnh đông người. Nói cách khác, đuôi độ trễ của bạn dày lên đúng ở cảnh mà bạn bật tầng ngữ nghĩa lên để xử lý. Một mô hình không triệt phi cực đại có thời gian chạy gần như hằng số, và với một bộ lập lịch thời gian thực thì hằng số quý hơn nhanh.

↪ **tiền đề** C/14/02 – cơ chế ghép cặp một-một nằm ở Chương 14; ở đây ta chỉ dùng hệ quả của nó về phân bố độ trễ

RT-DETR đẩy ý tưởng đó vào vùng thời gian thực [9]. Cơ chế là một bộ mã hoá lai: nó tách việc trộn thông tin trong cùng một mức độ phân giải khỏi việc trộn giữa các mức. Cách chọn truy vấn khởi đầu cũng tốt hơn bản gốc. Nó còn một tính chất hiếm và rất hợp với robot: có thể đổi tốc độ bằng cách bớt tầng giải mã lúc chạy, *không cần huấn luyện lại*. Với một hệ phải hạ chất lượng khi nóng máy hoặc khi tải cao, đó là một nút vặn có thật. Dòng RF-DETR năm 2025 đi tiếp bằng cách gắn một *backbone* tự giám sát mạnh vào khung DETR; các con số họ công bố tôi chưa kiểm chứng độc lập nên không dẫn lại ở đây.

D. SAM và SAM 2: phân đoạn bất kỳ, và chi phí thật của nó

SAM đổi câu hỏi từ “vật này là lớp gì” sang “vùng nào là một vật”, và trả lời theo prompt: một điểm bấm, một khung, hoặc một mặt nạ thô [10]. SAM 2 mở rộng sang video bằng một bộ nhớ chạy dọc thời gian, nên mặt nạ theo được vật qua các khung thay vì phải phân đoạn lại từ đầu [11].

Cấu trúc chi phí của SAM là bài học thiết kế đáng lấy đi kể cả khi bạn không dùng SAM. Bộ mã hoá ảnh rất nặng và chạy *một lần mỗi ảnh*. Bộ giải mã mặt nạ rất nhẹ và chạy *một lần mỗi prompt*. Vì thế hỏi thêm mười vùng nữa trên cùng một ảnh gần như miễn phí, trong khi chuyển sang ảnh mới thì trả lại toàn bộ giá. Đây đúng là mẫu “đặt phần đắt ở chỗ được khấu hao” mà bạn đã dùng khi tách trích xuất đặc trưng khỏi ghép cặp.

Nhưng có hai giới hạn phải nói thẳng, và cả hai đều chí mạng cho việc lọc vật động.

- **SAM không cho bạn nhãn.** Nó trả về các vùng, không trả về chữ “person”. Muốn biết vùng nào là vật có thể động, bạn vẫn cần một bộ phân loại hoặc một mô hình ngôn ngữ-thị giác chạy thêm. Chi phí thật vì thế là chi phí SAM cộng chi phí gán nhãn.
- **Chi phí bộ mã hoá không vừa ngân sách khung hình.** Bản gốc dùng một *transformer* thị giác cỡ lớn chạy toàn độ phân giải, tức dạng phép tính nặng nhất trong tất cả các công cụ mục này bàn tới. Các biến thể nhẹ của SAM 2 nhẹ hơn hẳn, nhưng vẫn không cùng hạng với một detector nhỏ. Tôi không nêu con số vì tôi không tự đo được nó trên phần cứng của bạn; phép đo đúng nằm ở dự án thứ hai cuối chương. Kết luận thực dụng thì không phụ thuộc con số: đây là công cụ ngoại tuyến, hoặc công cụ chạy *thừa* trên khung hình chính.

Tới 2026 dòng này đã có bản nhận prompt ở mức khái niệm và chạy trên video; tôi không kiểm chứng được chi tiết nên chỉ ghi nhận sự tồn tại của nó.

E. Grounding DINO và Open-YOLO 3D: khi tập nhãn phải mở

Grounding DINO nhận một câu chữ làm điều kiện và phát hiện vật theo mô tả, nhờ trộn đặc trưng ảnh với đặc trưng chữ ở nhiều tầng [12]. Nó gỡ đúng nút thắt tập nhãn đóng: nhà kho của bạn có xe đẩy hàng, pallet và tấm chắn, và COCO không có lớp nào cho chúng.

Cái giá là kích thước. Một bộ mã hoá chữ cộng một *backbone* nặng đặt nó vào cùng hạng nặng với SAM. Vai trò đúng của nó vì thế không phải trong vòng lặp. Nó là **bộ gán nhãn ngoại tuyến**: cho nó chạy chậm trên dữ liệu thu được, lấy nhãn ra, rồi chùng cất xuống một detector nhỏ chạy được thời gian thực. Mẫu “mô hình lớn dạy mô hình nhỏ” này là cách duy nhất tôi biết để có tập nhãn mở mà vẫn giữ được ngân sách.

Open-YOLO 3D là một ví dụ tốt của cùng tinh thần tiết kiệm, ở một tầng khác [13]. Bài toán là phân đoạn thực thể 3D với tập nhãn mở. Cách làm thông thường là chạy một mô hình thị giác-ngôn ngữ nặng trên từng khung rồi nâng kết quả lên 3D, và nó rất chậm. Open-YOLO 3D thay mặt nạ 2D bằng *hộp* 2D từ một detector mở nhẹ hơn, rồi dùng các hộp đó để bỏ phiếu nhãn cho các đề xuất 3D. Bài học thiết kế nằm ở chỗ này, và nó áp được sang chỗ khác: hỏi xem bạn thật sự cần độ chi tiết nào, rồi mua đúng độ chi tiết đó. Nếu việc chỉ cần biết “đám điểm này thuộc vật gì”, một cái hộp là đủ và mặt nạ từng điểm ảnh là lãng phí.

Bảng 1.1. Sáu công cụ nhận thức, đọc theo con mắt ngân sách. Cột sức nặng là một *thứ hạng định tính*, xếp theo hai thứ nhìn thấy được mà không cần chạy: kích thước mô hình, và dạng phép tính (một lượt trên ảnh nhỏ là nhẹ; một *transformer* thị giác chạy toàn độ phân giải là nặng). Thứ hạng này *không thay được phép đo*: sức nặng thật chỉ hiện ra khi bạn chạy mô hình cùng lúc với hệ SLAM trên chính máy của mình, theo quy trình ở dự án thứ hai cuối chương. Cột cuối là cột quyết định.

Công cụ	Cho bạn cái gì	Sức nặng (thứ hạng, không phải số đo)	Song song với theo vết?	Hóng khi nào
YOLO nhỏ (n/s, 640)	Hộp + nhãn, tập đóng	Nhẹ — mạng nhỏ, một lượt, ảnh nhỏ	Được, kể cả trên bộ tăng tốc chuyên dụng	Vật nhỏ, vật bị che, lớp ngoài COCO; đuôi độ trễ nếu còn triết phi cực đại
YOLO trung bình, RT-DETR nhẹ	Hộp + nhãn, độ trễ ổn định hơn	Trung bình — cùng dạng phép tính, mô hình lớn hơn	Chỉ ở tần số thấp hơn nhịp khung	Tranh chấp GPU với trích xuất đặc trưng; vẫn tập đóng
Phân đoạn thực thể nhẹ	Mặt nạ + nhãn	Trung bình — thêm một đầu ra dày đặc theo từng điểm ảnh	Khung hình chính thời	Biên mặt nạ lệch vài điểm ảnh — đúng chỗ góc ORB hay nằm
SAM	Vùng, <i>không</i> nhãn	Nặng — <i>transformer</i> thị giác cỡ lớn cho mỗi ảnh	Không	Không nói được vùng nào là vật động; phải cộng thêm bộ gán nhãn
SAM 2 (bản nhẹ)	Vùng + theo vết qua khung	Trung bình tới nặng, tùy biến thể	Chỉ trên khung hình chính, chạy thưa	Trôi mặt nạ khi vật bị che lâu; vẫn không có nhãn
Grounding DINO	Hộp theo mô tả chữ, tập mở	Nặng — cộng thêm cả một bộ mã hoá chữ	Không	Quá chậm cho vòng lặp; dùng làm bộ gán nhãn ngoại tuyến

CẠM BẤY

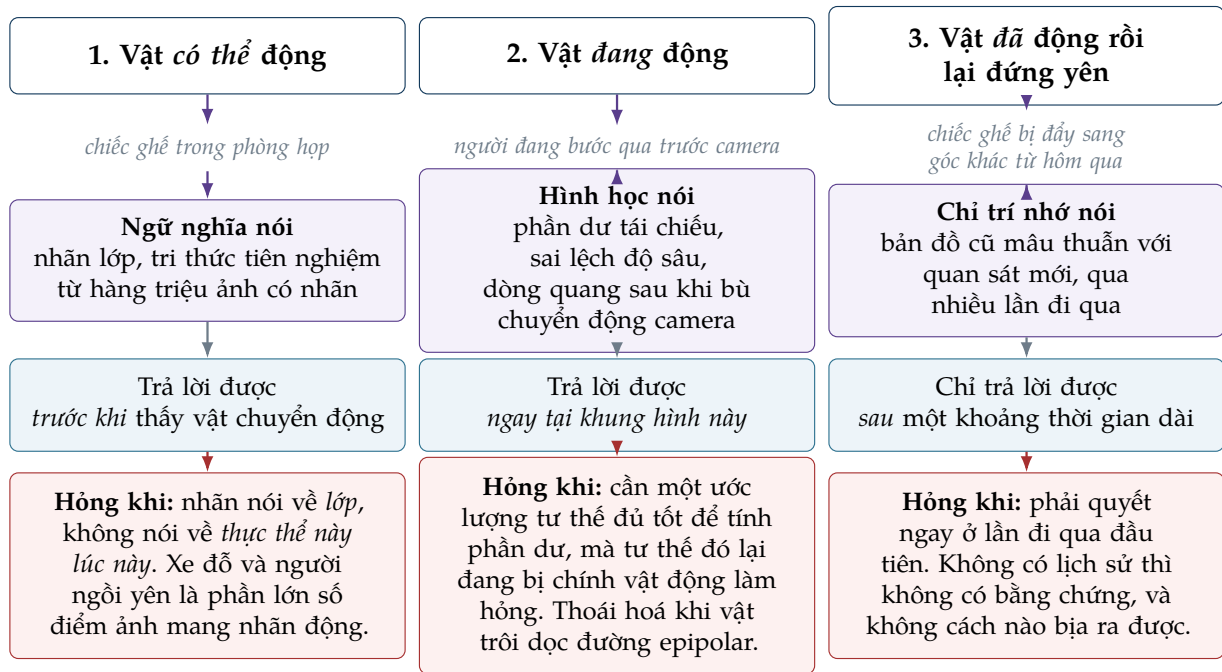
Bảng FPS trong paper gần như luôn đo trên máy để bàn có GPU rời, ảnh đơn lẻ, kích thước *batch* lớn, và *không* có tiến trình nào khác chạy cùng. Ba điều kiện đó đều sai với bạn. Chuyển sang Orin, *batch* bằng một, và có một hệ SLAM đang chiếm GPU, con số tụt đi rất xa — và không bảng nào cho bạn biết trước tụt bao nhiêu. Cách kiểm nhanh và trung thực: chạy mạng *cùng lúc* với hệ SLAM thật, đo p99 của cả hai, rồi mới nói tới trung bình.

1.3 Ba việc khác nhau khi loại vật thể động (Three Different Jobs)

Đây là mục cốt lõi của chương. Tài liệu về mảng này gộp ba việc rất khác nhau vào cùng một cụm “xử lý cảnh động”, và chính việc gộp đó khiến người đọc không thấy được vì sao một phương pháp hoạt động trên tập này mà chết trên tập kia. Tách ra thì mọi thứ sáng ra ngay.

A. Ba câu hỏi, ba nguồn bằng chứng

Ba việc là ba câu hỏi khác nhau về cùng một mảng điểm ảnh, và mỗi câu chỉ có đúng một nguồn bằng chứng trả lời được. Hình 1.3 là bản đồ của cả mục, và cũng là hình đáng nhớ nhất của chương.



Ba cột độc lập: không cột nào suy ra được cột nào. Một hệ chỉ có cột 1 xoá nhầm xe đỗ. Một hệ chỉ có cột 2 không khởi động được khi vật động chiếm phần lớn khung hình. Một hệ chỉ có cột 3 vô dụng ở lần chạy đầu tiên.

Hình 1.3. Ba việc khác nhau bị gộp dưới cùng một cái tên “xử lý cảnh động”. Mỗi cột có một nguồn bằng chứng riêng, một thời điểm mà nó trả lời được, và một kiểu hông riêng. Đây là bản đồ để đọc mọi paper trong mảng này: hãy hỏi paper đó đứng ở cột nào, và nó giả định hai cột kia đã được ai lo.

B. Cột 1 – vật có thể động, và ngữ nghĩa là nguồn duy nhất

Cột này là chỗ ngữ nghĩa thật sự không thể thay thế. Không có phép đo hình học nào trên hai khung hình liên tiếp nói được rằng cái ghế *có khả năng* bị đẩy, còn bức tường thì không. Tri thức đó đến từ nơi khác: từ việc mô hình đã xem hàng triệu ảnh trong đó ghế xuất hiện ở đủ chỗ còn tường thì không.

Giá trị của tri thức tiên nghiệm ấy là nó có *trước* khi có bằng chứng. Ở khung hình đầu tiên của chuỗi, bạn chưa có tư thế nào để tính phần dư, chưa có lịch sử nào để so. Nhưng bạn đã biết ngay rằng nửa dưới khung hình là một người. Điều đó cho phép hạ trọng số ngay lập tức, và giai đoạn khởi tạo là giai đoạn mong manh nhất của cả hệ.

Giới hạn cũng nằm ngay trong định nghĩa. Nhãn là một mệnh đề về *lớp*, không phải về *thực thể này ở khung hình này*. Trong phần lớn cảnh thật, đa số điểm ảnh mang nhãn “có thể động” lại đang đứng yên hoàn toàn: xe đỗ ven đường, người ngồi làm việc, ghế kê quanh bàn. Chúng là cấu trúc tốt, ổn định, giàu văn, và thường nằm ở khoảng cách vừa đẹp cho tam giác hoá. Xoá chúng là xoá dữ liệu tốt.

C. Cột 2 – vật đang động, và cái vòng luẩn quẩn của nó

Cột này thuộc về hình học, và bằng chứng thì có nhiều dạng. Bốn dạng hay dùng nhất, xếp theo độ mạnh của tín hiệu.

1. **Sai lệch độ sâu.** Chiếu một điểm bản đồ vào khung hiện tại, so độ sâu dự đoán với độ sâu đo được từ stereo hoặc RGB-D. Lênh quá ngưỡng nghĩa là vật đã dịch dọc trục nhìn. Đây là kiểm tra mạnh nhất khi có độ sâu tin cậy, và là kiểm tra mà DynaSLAM dùng [14].

2. **Khoảng cách tới đường epipolar.** Với hai khung và một tư thế tương đối, một điểm tĩnh phải nằm trên đường epipolar của nó. Lệch khỏi đường đó là bằng chứng chuyển động. Rẻ, không cần độ sâu, nhưng mù với chuyển động *dọc theo* đường epipolar.
3. **Dòng quang sau khi bù chuyển động camera.** Dự đoán trường dòng quang mà một cảnh tĩnh sẽ tạo ra dưới chuyển động camera đã ước lượng, rồi trừ khỏi dòng quang đo được. Phần dư khác không là chuyển động thật của vật. Đây là tín hiệu dày đặc nhất, và là chỗ *F/30-do-sau-dong-quang-va-mo-hinh-nen* nổi vào mục này.
4. **Tính nhất quán của một cụm điểm.** Nếu một nhóm điểm cùng vi phạm ràng buộc theo cùng một hướng, đó là một vật, không phải nhiễu. Đây là tín hiệu phân biệt vật động thật với ngoại lai ngẫu nhiên, và cũng là tín hiệu đắt nhất để tính.

Cả bốn đều mắc chung một vòng luẩn quẩn, và nói thẳng ra nó là điều trung thực nhất có thể nói về cột này. Muốn tính phần dư, bạn cần tư thế camera. Nhưng tư thế camera đang được ước lượng từ chính tập điểm chứa vật động. Khi vật động chiếm phần nhỏ khung hình, RANSAC gỡ được vòng này: nghiệm đồng thuận bám theo nền tĩnh, phần dư của vật động vọt lên, và bạn phát hiện được chúng. Khi vật động chiếm phần lớn khung hình, nghiệm đồng thuận bám theo *vật động*, phần dư của nền tĩnh vọt lên, và bạn phát hiện ngược hoàn toàn.

CẦN NHỚ

Đây là câu trả lời cho câu hỏi “RANSAC đã tốt rồi, mặt nạ thêm được gì”. RANSAC hỏng khi ngoại lai *đồng và nhất quán*, vì khi đó chúng thắng phiếu đồng thuận. Một đám đồng đi cùng chiều, một xe tải chạy song song, một bức tường người trong thang máy — đó chính xác là các trường hợp đó. Ngữ nghĩa hữu ích ở đúng chỗ này, và chỉ ở đây: nó phá thế bế tắc bằng một lá phiếu đến từ ngoài hình học. Ngoài vùng đó, nó chủ yếu chỉ lấy đi dữ liệu tốt. [T3]

Thoái hoá thứ hai đáng nhớ vì nó im lặng: vật di chuyển *dọc theo* tia nhìn hoặc dọc theo đường epipolar không tạo ra vi phạm nào cả. Một người đi thẳng ra xa camera trông y hệt một người đứng yên ở xa hơn một chút. Không kiểm tra hai khung nào bắt được trường hợp này. Chỉ có độ sâu đo trực tiếp, hoặc nhãn ngữ nghĩa, mới nói được.

D. Cột 3 – vật đã động rồi lại đứng yên, và chỉ trí nhớ nói được

Cột này bị bỏ quên nhiều nhất, dù với một robot dịch vụ nó lại là cột tồn kém nhất. Chiếc ghế hôm qua ở giữa phòng, hôm nay ở góc phòng. Ở cả hai lần đi qua, nó đứng yên tuyệt đối. Không phép kiểm tra hình học nào trong một lần chạy phát hiện được gì, vì trong lần chạy đó nó đứng là tĩnh. Nhãn ngữ nghĩa thì báo động ở cả hai lần, kể cả khi ghế không hề bị đụng tới.

Bằng chứng duy nhất là mâu thuẫn giữa bản đồ cũ và quan sát mới. Điểm bản đồ ở vị trí cũ liên tục không được quan sát lại dù nằm trong tầm nhìn và không bị che. Đó là tín hiệu “chỗ này không còn gì”. Nó đòi hỏi ba thứ mà một hệ SLAM một-lần-chạy không có: bản đồ sống qua nhiều phiên, cơ chế đếm số lần quan sát được và số lần đáng lẽ phải quan sát được, và một chính sách xoá điểm dựa trên tỉ số đó.

Đây chính là chỗ khái niệm “bản đồ theo thời gian” bước vào, và mục về đồ thị cảnh sẽ quay lại với Khronos. Điều đáng ghi nhớ ngay bây giờ: đây là bài toán *bảo trì bản đồ*, không phải bài toán nhận thức. Ném một mạng phân đoạn vào nó không giải quyết được gì.

E. DynaSLAM và họ hàng: các thiết kế đã có

Với ba cột trong tay, đọc các hệ thống có sẵn trở nên nhanh. Mạch dưới đây đi theo thứ tự lịch sử, và mỗi bước gỡ đúng một nút thắt của bước trước.

I Mạch — từ mặt nạ tới bản đồ bốn chiều

- 1 Vấn đề** ORB-SLAM2 chạy trên chuỗi có người đi ngang thì quỹ đạo lệch hẳn, dù mọi thứ khác đều đúng. Nguyên nhân là các điểm trên người vẫn được đưa vào tối ưu như điểm tĩnh.

Giải pháp DynaSLAM đặt một mạng phân đoạn thực thể trước bộ theo vết, xoá mọi đặc trưng rơi vào mặt nạ của các lớp có thể động, rồi chạy thêm một kiểm tra hình học đa khung cho phần còn lại [14].

Giả định cột 1 và cột 2 bổ sung cho nhau, và cái nào bắt được thì bắt.

Giá phải trả một lượt mạng nặng cho mỗi khung, và mất toàn bộ đặc trưng trên các vật đứng yên thuộc lớp động.
- 2 Vấn đề** kiểm tra hình học của DynaSLAM cần độ sâu, nên bản đơn ảnh chỉ còn lại mặt nạ.

Giải pháp DS-SLAM ghép một mạng phân đoạn nhẹ với một kiểm tra nhất quán chuyển động dựa trên dòng quang và ràng buộc epipolar, chạy trên hai luồng song song [15].

Hệ quả đây là bản thiết kế đầu tiên coi tầng ngữ nghĩa là một luồng riêng thay vì một bước trong đường tới hạn.

Giá phải trả mặt nạ trễ một khung, và kiểm tra epipolar mù với chuyển động dọc đường epipolar.
- 3 Vấn đề** xoá vật động là vứt bỏ thông tin; vật động cũng là một vật có tư thế đáng ước lượng.

Giải pháp nhóm hợp nhất theo vật — MaskFusion và họ hàng — dựng một mô hình 3D riêng cho từng vật và theo vết tư thế của nó song song với tư thế camera [16]. DynaSLAM II đưa hẳn tư thế vật vào bài toán hiệu chỉnh chùm tia, nên vật động trở thành ràng buộc chứ không còn là nhiễu [17].

Giả định mỗi vật là một khối cứng.

Giá phải trả giả định khối cứng sai với con người, tức là với loại vật động phổ biến nhất; chi phí tính toán tăng theo số vật.
- 4 Vấn đề** mọi thứ ở trên chỉ sống trong một lần chạy, nên cột 3 vẫn bỏ ngỏ.

Giải pháp Khronos phát biểu lại bài toán theo trục thời gian: thay vì phân loại tĩnh hay động, hệ ước lượng *khoảng thời gian tồn tại* của từng vật, nên xử lý được cả chuyển động ngắn hạn lẫn thay đổi dài hạn trong cùng một khung [18].

Hệ quả bản đồ trả lời được câu hỏi “thế giới trông thế nào lúc t ” thay vì chỉ “thế giới trông thế nào”.

Giá phải trả cần một stack bản đồ đặc và cần lưu lịch sử, hai thứ mà một hệ điểm thưa không có.

F. Vì sao mặt nạ ngữ nghĩa một mình là không đủ

Gom lại thành sáu lý do độc lập. Mỗi lý do là một chế độ hỏng riêng, và một hệ thật thường gặp vài lý do cùng lúc.

- **Nhân là mệnh đề về lớp, không phải về thực thể.** Đây là lý do gốc, và năm lý do sau chỉ là hệ quả ở các tầng khác nhau. Xe đỗ, người ngồi, ghế kê sát tường đều mang nhãn động mà không hề động.

- **Vật động ngoài tập nhãn.** Xe đẩy hàng, cánh cửa, rèm cửa, thùng hàng được bê đi, con chó. Tám mươi lớp của COCO không phủ được một nhà kho hay một bệnh viện. Tập nhãn mở giải được vấn đề này về nguyên tắc, nhưng với chi phí đã nêu ở mục trước.
- **Chuyển động gián tiếp.** Một vật tĩnh bị vật động làm đổi vẻ ngoài: bóng đổ chạy trên tường, ánh phản chiếu trên sàn bóng và trên kính, một cái ghế bị người đẩy. Điểm ảnh nằm trên bề mặt tĩnh, nên không mặt nạ nào phủ chúng, nhưng phép đo thì sai.
- **Biên mặt nạ lệch đúng chỗ đau nhất.** Mạng phân đoạn sai vài điểm ảnh ở biên. Mà góc ORB thì thích nằm đúng ở biên, vì đó là chỗ gradient mạnh nhất. Bạn buộc phải chọn: nở mặt nạ ra và xoá thêm cấu trúc tĩnh, hay giữ nguyên và để lọt đúng những điểm tệ nhất — các điểm nằm trên đường viền vật động.
- **Mặt nạ luôn cũ.** Nó được tính trên khung k và đến nơi ở khung $k + 2$ hoặc muộn hơn. Sai lệch vị trí tỉ lệ với tốc độ vật, nên nó lớn nhất đúng ở trường hợp mặt nạ đáng giá nhất. Truyền tiếp bằng dòng quang hoặc theo vết hợp chỉ giảm chứ không xoá được sai lệch này.
- **Mạng hồng đúng lúc bạn cần nó.** Ảnh mờ do chuyển động, thiếu sáng, phơi sáng nhảy — đó vừa là những khung hình mà hình học yếu nhất, vừa là những khung hình nằm ngoài phân phối huấn luyện của mạng. Hai kiểu hồng *tương quan dương* với nhau. Một hệ dự phòng chỉ có giá trị khi nó hồng độc lập với hệ chính, và ở đây thì không.

Lý do cuối đáng được nhấn mạnh vì nó là kết luận thiết kế, không phải một chi tiết kỹ thuật. Bạn không thể coi tầng ngữ nghĩa là lưới an toàn cho các khung hình khó. Nó là một tầng hỗ trợ chỉ hoạt động tốt ở đúng những khung hình mà hình học vốn đã ổn.

↪ cùng nguyên lý với B/08 — đây là dịch chuyển phân phối ở dạng độc nhất: phân phối trượt đi đúng theo hướng làm cả hai thành phần của hệ cùng yếu đi

G. Hai phép tính nhỏ cho thấy vùng an toàn hẹp tới đâu

Trước khi bàn tới cái giá của việc xoá, hãy làm hai phép tính bằng tay. Cả hai chỉ dùng số liệu bạn đã có, và cả hai đều cho ra kết quả hẹp hơn trực giác.

Phép tính thứ nhất: một người đứng gần chiếm bao nhiêu phần khung hình. Đây là một phép tính *giả định*, làm để thấy hình dạng của quan hệ chứ không phải để lấy con số. Giả sử một camera kiểu EuRoC, tiêu cự khoảng 460 điểm ảnh, ảnh 752×480 , tức 360 960 điểm ảnh. Một người rộng chừng 0,5 m và cao 1,7 m. Ở khoảng cách 3 m, bề rộng chiếu xuống ảnh là $460 \times 0,5/3 \approx 77$ điểm ảnh và bề cao là $460 \times 1,7/3 \approx 261$. Diện tích chừng 20 000 điểm ảnh, tức *khoảng 6%* khung hình. Vô hại.

Bây giờ cũng người đó ở khoảng cách 1 m. Bề rộng thành $460 \times 0,5 \approx 230$ điểm ảnh, bề cao thành 782 nên bị cắt còn 480. Diện tích là $230 \times 480 = 110\,400$, tức *khoảng 31%* khung hình. Cùng một người, chỉ đổi khoảng cách ba lần, mà tỉ lệ nhảy năm lần. Điều đáng mang đi không phải hai con số mà là quan hệ: diện tích đi theo nghịch đảo bình phương khoảng cách, nên phân bố diện tích vật động có đuôi rất dày. Hệ quả thực dụng là bạn phải đo ở một phân vị cao chứ không ở trung bình — trung bình của cả chuỗi có thể rất nhỏ trong khi đúng vài chục khung hình quyết định lại rất lớn, và chính vài chục khung đó làm hồng quỹ đạo.

Phép tính thứ hai: RANSAC chịu được tới đâu. Số vòng lặp cần để lấy được một mẫu sạch với xác suất p là

$$N = \frac{\log(1 - p)}{\log(1 - (1 - \varepsilon)^s)},$$

với ε là tỉ lệ ngoại lai và s là số điểm tối thiểu mỗi mẫu. Lấy $p = 0,99$ và $s = 4$, rồi thay vài giá trị:

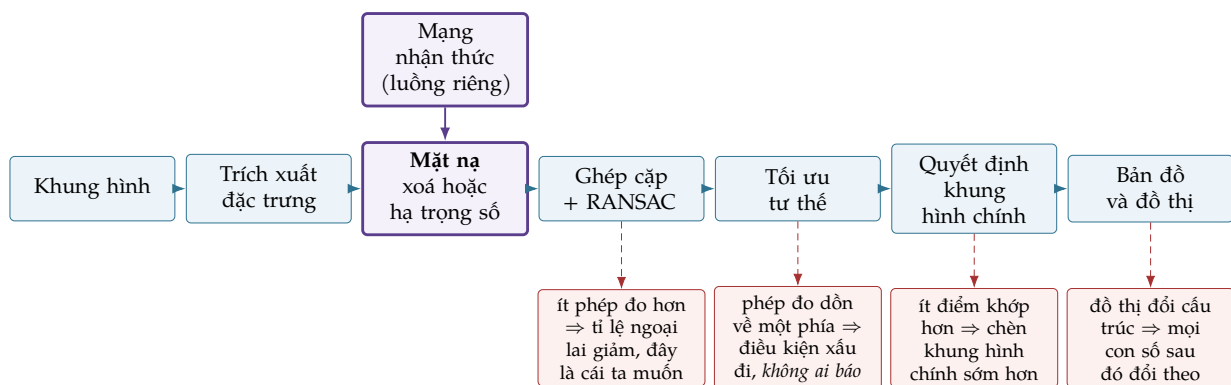
- $\varepsilon = 0,20 \rightarrow (0,8)^4 = 0,41 \rightarrow N \approx 9$ vòng. Rẻ như không.
- $\varepsilon = 0,50 \rightarrow (0,5)^4 = 0,0625 \rightarrow N \approx 72$ vòng. Vẫn chạy được.

- $\varepsilon = 0,70 \rightarrow (0,3)^4 = 0,0081 \rightarrow N \approx 566$ vòng. Bắt đầu đau.
- $\varepsilon = 0,85 \rightarrow (0,15)^4 = 5,1 \times 10^{-4} \rightarrow N \approx 9\,000$ vòng. Hết cửa.

Chi phí bùng nổ theo hàm mũ của s , nhưng đó chưa phải phần đáng sợ nhất. Công thức trên giả định ngoại lai là *ngẫu nhiên*, nên chúng không bao giờ đồng thuận với nhau. Vật động thì không ngẫu nhiên: cả đám điểm trên một người cùng vi phạm theo cùng một hướng. Khi tỉ lệ đó vượt một nửa, tập điểm động trở thành *nhóm đa số nhất quán*, và RANSAC chọn đúng chúng làm nghiệm. Lúc đó nó không chỉ chậm; nó cho ra câu trả lời sai một cách tự tin.

Ghép hai phép tính lại thì ra vùng làm việc thật của tầng ngữ nghĩa, và nó hẹp. Khi vật động chỉ chiếm một phần nhỏ khung hình, RANSAC gánh được và mặt nạ gần như không thêm gì. Khi vật động chiếm quá nửa số phép đo và cùng vi phạm theo một hướng, chúng thành nhóm đa số nhất quán, RANSAC lật ngược, và mặt nạ là thứ duy nhất cứu được — ranh giới “quá nửa” này suy từ chính cơ chế bỏ phiếu, không phải từ một ngưỡng chính tay. Giữa hai đầu là một vùng xám, và chỗ nó bắt đầu với chỗ nó kết thúc thì phụ thuộc cảnh, phụ thuộc mật độ đặc trưng và phụ thuộc chính hệ của bạn. Đó là thứ phải *đo*, không phải thứ đọc ra từ một con số in sẵn, và thí nghiệm oracle ở mục I là cách đo nó.

Hình 1.4 cho thấy mặt nạ cắm vào chỗ nào trong đường ống, và quan trọng hơn, nó đổi những gì ở phía sau. Chú ý rằng nó không chỉ đổi tập phép đo: nó đổi luôn quyết định chèn khung hình chính, và quyết định đó lan ra toàn bộ đồ thị.



Mặt nạ can thiệp ở một chỗ, nhưng hậu quả lan ra bốn chỗ. Chỉ hậu quả đầu tiên là thứ bạn định làm. Ba hậu quả sau là tác dụng phụ, và chúng là lý do một so sánh “có mặt nạ với không mặt nạ” hiếm khi cô lập được đúng biến bạn muốn đo.

Hình 1.4. Mặt nạ cắm vào đâu và nó đổi những gì. Điểm can thiệp chỉ có một, ngay sau trích xuất đặc trưng, nhưng hậu quả lan qua bốn khối phía sau. Hậu quả thứ hai là kiểu hồng im lặng, và hậu quả thứ ba giải thích vì sao mọi so sánh trong mảng này đều cần khoá hoặc báo cáo chính sách khung hình chính.

H. Vì sao xoá quá tay còn tệ hơn không xoá

Đây là điểm mà tôi cho là quan trọng nhất của cả chương, và cũng là điểm ít được viết ra nhất trong các paper về mảng này. Trực giác thông thường nói rằng xoá thừa thì cùng lắm là lãng phí. Trực giác đó sai, vì nó coi các phép đo là những mẫu bằng chứng độc lập và có thể thay thế cho nhau. Chúng không phải vậy.

Bộ tối ưu không cần *nhiều* ràng buộc. Nó cần ràng buộc *trải đủ mọi hướng*. Một mặt nạ xoá đi một mảng lớn liền khối của ảnh — và mặt nạ ngữ nghĩa luôn liền khối, vì vật thể thì liền khối. Các điểm còn lại vì thế dồn về phần còn lại của khung hình. Ma trận thông tin của bài toán tư thế mất đi một hướng, và tư thế có một chiều gần như không bị ràng buộc.

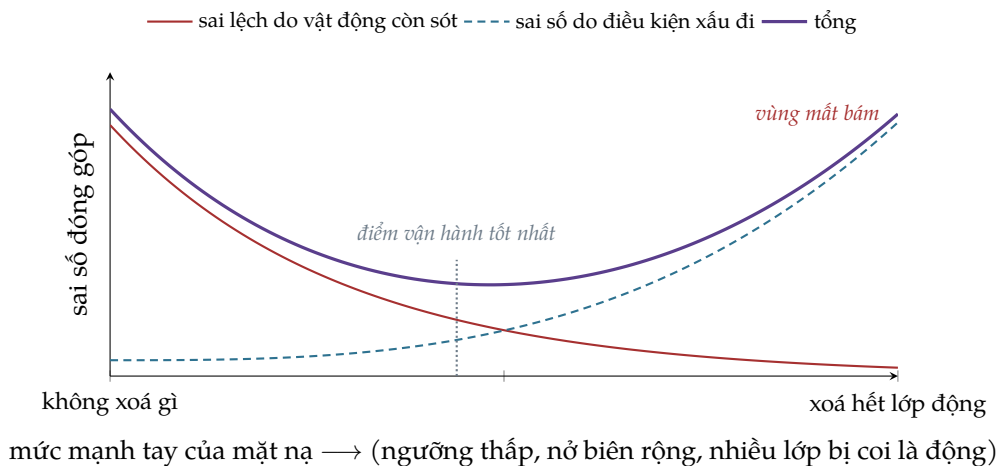
Kiểu hồng này im lặng, và đó là chỗ nguy hiểm. Không có cớ ngoại lai nào bật lên. Phần dư còn lại nhỏ và đẹp, vì mọi phép đo còn lại đều là phép đo tốt. Hiệp phương sai mà hệ báo ra thì lạc

quan, vì nó tính từ chính tập điểm đã bị lọc. Sai số chỉ hiện ra sau đó vài giây, dưới dạng trôi, và lúc đó bạn sẽ đi tìm nguyên nhân ở chỗ khác.

Ba cái giá cụ thể, xếp theo mức độ hay gặp:

- **Mất bám hần.** Hệ tuyên bố mất bám khi số điểm bản đồ theo được tụt dưới ngưỡng. Trên một hành lang trắng, xoá một người đứng gần camera có thể lấy đi phần lớn số đặc trưng còn lại. Cái giá không phải vài xăng-ti-mét sai số; nó là một lần định vị lại, và định vị lại có thể chọn nhầm chỗ.
- **Đổi chính sách khung hình chính.** Ít điểm khớp hơn làm điều kiện chèn khung hình chính bật sớm hơn. Cấu trúc đồ thị đổi, mật độ khung hình chính đổi, và mọi con số sau đó đổi theo. Khi bạn so hai cấu hình mà một bên có mặt nạ, bạn thường đang so hai đồ thị khác nhau chứ không phải hai bộ phép đo khác nhau.
- **Điều kiện xấu đi mà không ai báo.** Trường hợp đã mô tả ở trên. Nó là kiểu hỏng khó chẩn đoán nhất vì mọi chỉ số nội bộ đều trông khoẻ.

Hình 1.5 vẽ hai cái giá đối nghịch nhau theo mức độ mạnh tay của mặt nạ. Đây là hình minh hoạ dạng hàm, không phải số đo: hình dạng của hai đường thì tôi tin, còn vị trí cực tiểu thì phụ thuộc cảnh và phải tự đo trên dữ liệu của bạn.



Hình 1.5. Hai cái giá đối nghịch. Xoá ít thì vật động còn sót lại và tạo sai lệch có hướng. Xoá nhiều thì các điểm còn lại dồn về một phía và điều kiện của bài toán tư thế xấu đi; đường này dốc lên nhanh vì nó phi tuyến, và ở cuối nó chuyển thành mất bám hần. Đường tổng có cực tiểu ở giữa, và cực tiểu đó *phụ thuộc cảnh*: hành lang trắng đẩy nó sang trái, quảng trường đông người đẩy nó sang phải. Hình minh hoạ dạng hàm, không phải số đo.

Từ hình đó rút ra hai kết luận thực dụng. Thứ nhất, **hãy hạ trọng số thay vì xoá**. Một mặt nạ nhị phân là một quyết định cứng đặt ở chỗ tồi nhất có thể: đặt ngay đầu đường ống, trước khi bất kỳ bằng chứng nào được thu thập. Nhân độ tin cậy của mặt nạ vào ma trận thông tin của phép đo thì mềm hơn nhiều. Điểm nghi ngờ vẫn đóng góp vào điều kiện của bài toán, nhưng đóng góp ít. Nếu nó thật sự là ngoại lai, hàm mất mát bền vững sẽ đập nó ở vòng lặp sau, khi đã có bằng chứng thật.

↪ **cùng nguyên lý với F/34** — đây đúng là luận điểm của chương 34: mọi quyết định “tin hay không tin” trong đồ thị nhân tử cuối cùng đều là một con số trong ma trận thông tin, và biến nó thành nhị phân là vứt đi thông tin

Kết luận thứ hai ngắn hơn. **Điểm vận hành phải phụ thuộc vào số điểm còn lại**, chứ không phải là một hằng số chỉnh một lần. Quy tắc mà tôi thấy hợp lý — và nói rõ đây là suy luận thiết kế chứ chưa phải kết quả đo — là để mặt nạ mạnh tay khi số đặc trưng còn lại vẫn dư dả, rồi tự động

nơi lỏng khi số đó tụt về gần ngưỡng mất bám. Hệ nên chấp nhận vài phép đo bản hơn là chấp nhận mất bám, vì mất bám đắt hơn nhiều bậc.

I. Đo cho đúng, nếu không bạn sẽ tin nhầm

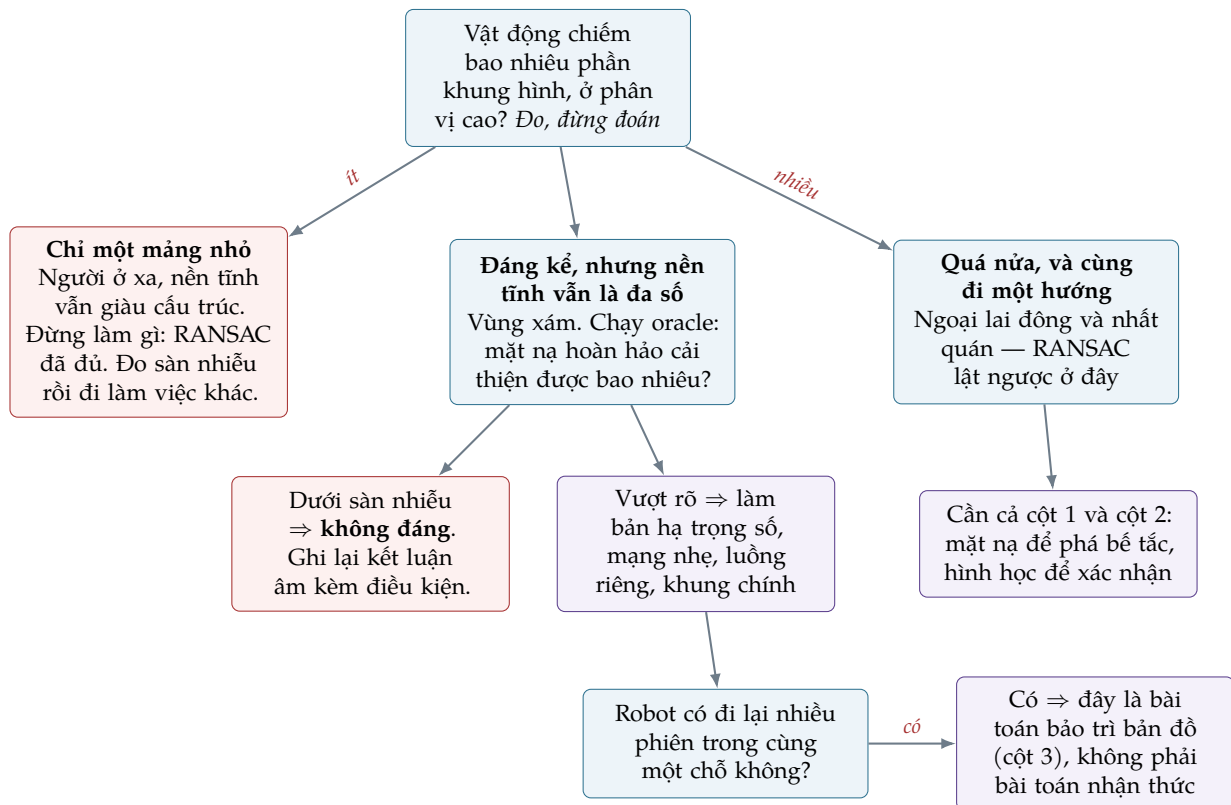
Mảng này có một vấn đề đánh giá nghiêm trọng, và biết trước nó sẽ giúp bạn đọc mọi bảng kết quả trong mảng với đúng liều hoài nghi.

Các con số “cải thiện rất lớn” trong mảng này phần lớn được báo trên vài chuỗi RGB-D trong nhà, nơi có một hoặc hai người chiếm phần rất lớn khung hình, và đối chứng là một hệ hoàn toàn không xử lý cảnh động [19]. Kể cả khi các con số đó thật, chúng gần như vô nghĩa với robot của bạn, vì chúng đo một chế độ hồng thảm khốc trong một cấu hình cực đoan. Robot dịch vụ đi hành lang gặp người chiếm một phần nhỏ khung hình, và ở chế độ đó thì RANSAC vốn đã xử lý ổn.

Bốn kỷ luật đo, và cả bốn đều là thứ bạn đã dùng ở phía SLAM, chỉ cần bê nguyên sang.

1. **Báo tỉ lệ hoàn thành cùng với APE, luôn luôn.** Một hệ mất bám giữa chừng có thể có APE rất đẹp trên đoạn nó còn sống. Với mặt nạ ngữ nghĩa, đây không phải rủi ro lý thuyết mà là chế độ hồng chính. Xem B/07-chia-du-lieu-va-danh-gia cho cách đọc cặp số này.
2. **Đo sàn nhiều trước.** Chạy cùng một cấu hình nhiều lần và lấy độ lệch chuẩn, rồi mới nói về cải tiến [20]. Bạn đã biết hệ của mình là phi tất định; thêm một mạng vào không làm nó tất định hơn.
3. **Chạy một thí nghiệm oracle trước khi làm gì cả.** Gán mặt nạ bằng tay, hoặc bằng một mô hình rất nặng chạy ngoại tuyến, trên vài trăm khung hình. Đó là mặt nạ hoàn hảo. Mặt nạ hoàn hảo cho bạn cái *trần*: mọi công sức làm mặt nạ chạy thời gian thực đều bị chặn trên bởi đúng con số đó. Nếu trần ấy không vượt nổi sàn nhiều bạn đã đo, hãy dừng và đi làm việc khác. Đây là công cụ có giá trị cao nhất trong cả mục.
4. **Cô lập yếu tố gây nhiễu về khung hình chính.** So sánh có mặt nạ với không mặt nạ thường đồng thời đổi mật độ khung hình chính. Muốn biết mặt nạ đóng góp gì, hãy khoá chính sách khung hình chính lại, hoặc báo cáo số khung hình chính kèm theo.

Hình 1.6 gói cả mục thành một cây quyết định, và câu trả lời phổ biến nhất trên cây đó là “chưa cần”.



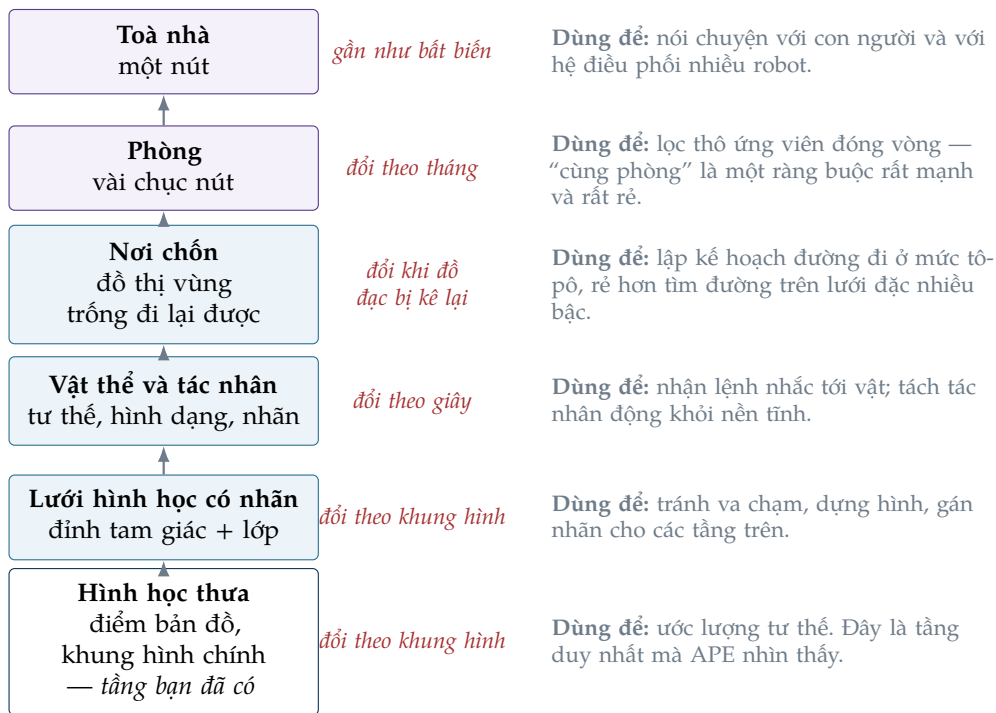
Hình 1.6. Cây quyết định cho tầng ngữ nghĩa. Các nhánh được mô tả bằng *tình huống* chứ không bằng ngưỡng số, vì ngưỡng đúng phụ thuộc mật độ đặc trưng của cảnh và phải tự do. Nhánh trái cùng là nhánh hay đúng nhất và ít được chọn nhất. Chú ý rằng nhánh phải cùng dẫn tới một bài toán khác hẳn — bảo trì bản đồ qua nhiều phiên — mà không mạng nhận thức nào giải được.

1.4 Đồ thị cảnh 3D (3D Scene Graphs)

Tới đây ta đổi câu hỏi. Nửa đầu chương hỏi ngữ nghĩa giúp *ước lượng tư thế* được gì. Nửa sau hỏi ngữ nghĩa cho *bản đồ* thêm được gì, và người tiêu thụ bản đồ ở đây không còn là script tính APE. Điểm xuất phát là một quan sát về biểu diễn. Bản đồ của ORB-SLAM3 là một danh sách phẳng gồm điểm 3D và khung hình chính. Nó hoàn hảo cho việc nó sinh ra để làm: cung cấp ràng buộc cho bộ tối ưu. Nhưng nó không trả lời được bất kỳ câu hỏi nào khác. “Tôi đang ở phòng nào” không có câu trả lời, vì không có khái niệm phòng. “Cái ghế nào gần tôi nhất” cũng vậy. Đồ thị cảnh 3D là câu trả lời cho câu hỏi: nếu ta xếp thêm vài tầng trừu tượng lên trên đám mây điểm đó thì được gì.

A. Cấu trúc tầng, và mỗi tầng mua được gì

Hình 1.7 vẽ cấu trúc mà dòng Kimera và Hydra dùng, cùng tốc độ thay đổi và người tiêu thụ của từng tầng [21], [22]. Cột bên phải là cột đáng đọc nhất: nó cho thấy các tầng trên không phải trang trí, chúng có người dùng thật.



Khi một vòng đóng lại, toàn bộ các tầng phải biến dạng theo, không chỉ đồ thị tư thế. Đó là bài toán kỹ thuật khó nhất của Hydra, và là lý do một đồ thị cảnh không phải chỉ là “bản đồ cộng nhãn”.

Hình 1.7. Sáu tầng của một đồ thị cảnh 3D, kèm tốc độ thay đổi và người tiêu thụ. Tốc độ thay đổi giảm dần khi lên cao, và đó chính là giá trị của cấu trúc tầng: các tầng trên ổn định nên dùng được làm mỏ neo, trong khi tầng dưới thì đổi liên tục. Tầng dưới cùng là thứ hệ SLAM của bạn đã có; năm tầng trên là thứ phải trả thêm để có.

B. Kimera: thư viện, và bài học về ranh giới mô-đun

Ý tưởng gắn nhãn ngữ nghĩa lên một bản đồ đặc không mới; SemanticFusion đã làm điều đó từ 2017 bằng cách hợp nhất dự đoán từng khung vào một bản đồ bề mặt [23]. Cái Kimera thêm vào là tính mô-đun và tính thời gian thực. Kimera là một thư viện mã nguồn mở dựng bản đồ đo đạc kèm ngữ nghĩa theo thời gian thực, gồm bốn mô-đun tách rời: một bộ đo quán tính–thị giác, một bộ tối ưu đồ thị tư thế bền vững, một bộ dựng lưới cục bộ nhanh, và một bộ gắn nhãn ngữ nghĩa lên lưới toàn cục [24].

Với bạn, giá trị lớn nhất của Kimera không nằm ở thuật toán nào cả. Nó nằm ở chỗ bốn mô-đun ấy có giao diện rõ ràng và thay được từng cái. Bạn có thể giữ nguyên bộ đo quán tính–thị giác của mình và chỉ mượn phần dựng lưới có nhãn. Rất ít hệ thống trong Phần F cho phép mượn từng phần như vậy; phần lớn là một khối liền không tháo rời được. Đó là một bài học thiết kế đáng lấy đi kể cả khi bạn không dùng dòng code nào của Kimera.

Bộ tối ưu đồ thị tư thế bền vững cũng đáng chú ý riêng. Nó loại vòng sai bằng một cơ chế bền vững tăng dần thay vì bằng một ngưỡng cứng, và đó là một câu trả lời trực tiếp cho câu hỏi “một vòng sai đắt bao nhiêu” của F/29-dinh-vi-lai-va-dong-vong.

C. Hydra: dựng đồ thị theo thời gian thực, và biến dạng khi vòng đóng

Hydra là bước từ “đồ thị cảnh như một sản phẩm hậu xử lý” sang “đồ thị cảnh dựng tăng dần trong lúc chạy” [22]. Ba đóng góp, và cái thứ hai là cái khó nhất.

- **Dựng tăng dần.** Các tầng được cập nhật liên tục từ dữ liệu cảm biến, chứ không dựng lại từ đầu. Phân đoạn phòng chạy trên đồ thị nơi chôn theo lối hình học chứ không cần một mạng riêng.
- **Biến dạng toàn đồ thị khi đóng vòng.** Đây là chỗ khó. Một đồ thị tư thế chỉ cần chỉnh lại vài chục tư thế. Một đồ thị cảnh phải kéo theo lưới hàng trăm nghìn đỉnh, các vật thể, các nơi chôn và ranh giới phòng, và phải làm trong thời gian thực. Hydra dùng một đồ thị biến dạng nhúng để lan hiệu chỉnh xuống các tầng dưới. Nếu bạn từng viết phần cập nhật bản đồ sau một lần tối ưu toàn cục, bạn biết đây là loại việc dễ sinh bết tắc và dễ sinh trạng thái không nhất quán.
- **Phát hiện vòng từ chính cấu trúc phân tầng.** Thay vì so một bộ mô tả ảnh toàn cục, Hydra so từ trên xuống: cấu hình phòng, rồi chòm sao vật thể, rồi mới tới vẻ ngoài. Đây là ý đáng suy nghĩ nhất của cả mục đối với nỗi đau đóng vòng, vì nó có *chế độ hồng khác* với bộ mô tả vẻ ngoài. Bộ mô tả vẻ ngoài chết vì đối ánh sáng và vì hành lang giống nhau; chòm sao vật thể thì không quan tâm tới ánh sáng, nhưng lại chết ở nơi trống trải không có vật nào.

Hydra-Multi mở rộng sang nhiều robot cùng dựng chung một đồ thị, với một nút tập trung lo việc gộp và lo các vòng liên robot [25]. Vấn đề kỹ thuật thật nằm ở gộp: hai robot gọi cùng một cái phòng bằng hai nút khác nhau, và phải quyết định khi nào chúng là một.

D. Clio: tác vụ quyết định mức chi tiết

Clio đặt một câu hỏi mà tôi cho là câu hỏi hay nhất trong cả mảng đồ thị cảnh: *cái gì được tính là một vật thể?* [26]

Câu trả lời thông thường lấy từ tập nhãn: một vật là một thứ có tên trong danh sách lớp. Clio nói câu trả lời đó sai về nguyên tắc, vì “vật thể” không phải thuộc tính của cảnh, nó là thuộc tính của *tác vụ*. Một chồng sách là một vật nếu nhiệm vụ là “dọn chồng sách này đi”. Nó là mười vật nếu nhiệm vụ là “tìm quyển bìa đỏ”. Không có mức chi tiết nào đúng khi chưa biết nhiệm vụ.

Cơ chế Clio dùng để hình thức hoá điều đó là nút thất thông tin: cho trước một danh sách nhiệm vụ viết bằng ngôn ngữ tự nhiên, gom các mảnh nguyên thuỷ thành các cụm sao cho cụm giữ được nhiều thông tin nhất về nhiệm vụ trong khi vẫn đơn giản nhất có thể. Bản đồ vì thế *co giãn theo nhiệm vụ*: cùng một cảnh cho ra hai đồ thị khác nhau với hai danh sách nhiệm vụ khác nhau.

Bài học thiết kế ở đây rộng hơn đồ thị cảnh rất nhiều, và nó nổi thẳng vào thói quen của bạn. Đó chính là nguyên tắc “chọn metric khớp chi phí thật” đặt vào tầng biểu diễn: mức chi tiết của bản đồ phải suy ra từ việc bản đồ sẽ được dùng để làm gì, chứ không suy ra từ việc cảm biến cho được bao nhiêu.

→ cùng nguyên lý với B/05 — đây là bước “phát biểu bài toán từ chi phí thật” của chương thiết kế bài toán, lần này áp cho biểu diễn bản đồ chứ không cho hàm mất mát

E. Khronos: thời gian là một trục, không phải một cái cờ

Khronos quay lại đúng cột 3 của mục trước, và nó là câu trả lời nghiêm túc nhất tôi biết cho cột đó [18]. Ý tưởng cốt lõi là bỏ hẳn phép phân loại nhị phân tĩnh hay động. Thay vào đó, mỗi vật được gán một *khoảng thời gian tồn tại*: nó xuất hiện ở đâu, từ lúc nào tới lúc nào.

Phát biểu đó gộp được hai hiện tượng vốn bị coi là khác loại. Một người đi ngang là một vật có khoảng tồn tại rất ngắn ở mỗi vị trí. Một chiếc ghế bị kê lại là một vật có khoảng tồn tại dài ở vị trí cũ, rồi một khoảng khác ở vị trí mới. Cùng một mô hình, cùng một cơ chế suy luận. Bản đồ kết quả trả lời được câu hỏi “thế giới trông thế nào lúc t ”, và với robot đi lại lâu dài thì đó mới là câu hỏi đúng.

Cái giá thì thẳng thắn. Bạn cần bản đồ đặc, cần lưu lịch sử, và cần đủ nhiều lần đi qua để có bằng chứng. Với một hệ điểm thưa chạy một lần trên một chuỗi, không có gì để lấy về ở đây cả.

F. ConceptGraphs và EFM3D: tập nhãn mở, và một cái thước

ConceptGraphs dựng đồ thị cảnh với tập nhãn mở bằng cách ghép ba mảnh có sẵn: một bộ phân đoạn không phân biệt lớp cho ra vùng, một mô hình thị giác-ngôn ngữ gán đặc trưng cho từng vùng, và một mô hình ngôn ngữ suy ra quan hệ giữa các vật để làm cạnh [27]. Không huấn luyện gì thêm.

Đây là kiến trúc rất hợp thời và cũng rất đắt. Mỗi khung hình phải qua ít nhất hai mô hình nền, và chất lượng đồ thị bị chặn trên bởi chất lượng của bước liên kết vùng qua các khung — đúng cái bước mà không mô hình nền nào lo hộ bạn. Với ngân sách Orin, đây là công cụ ngoại tuyến.

EFM3D không phải một hệ thống mà là một cái thước: một benchmark cho các mô hình nền 3D nhìn từ góc nhìn người đeo thiết bị, với hai tác vụ là phát hiện vật thể 3D và hồi quy bề mặt [28]. Tôi nhắc nó ở đây vì một lý do đã lặp lại nhiều lần trong Phần F: trong một mảng chưa chín, một cái thước tốt thường đẩy tiến bộ đi xa hơn một hệ thống mới. Mảng đồ thị cảnh vẫn đang thiếu thước đo chung, và đó là một phần lý do khó so sánh các công trình trong mảng.

G. Cái giá thật của việc nhận một đồ thị cảnh vào hệ của bạn

Cần nói rất rõ chỗ này, vì nó dễ bị đánh giá thấp. Nhận một đồ thị cảnh không phải là “thêm một mạng”. Nó là đổi loại bản đồ.

Mọi hệ trong mục này đều dựng trên một bản đồ *đặc*: lưới tam giác hoặc lưới voxel, cập nhật liên tục từ độ sâu. ORB-SLAM3 không có thứ đó và không có chỗ để cắm vào. Nhận đồ thị cảnh nghĩa là phải thêm một luồng dựng bản đồ đặc, và luồng đó tiêu bộ nhớ cùng băng thông ở bậc hoàn toàn khác với một đám mây điểm thưa. Trên Orin, đây thường là ràng buộc chặn chứ không phải chi phí mạng nhận thức.

Kèm theo đó là một khoản nợ kỹ thuật ít ai nhắc: mỗi lần tối ưu toàn cục, tất cả các tầng phải được kéo theo cho nhất quán. Bạn đã từng gặp bế tắc giữa luồng lập bản đồ cục bộ và luồng hiệu chỉnh toàn cục. Số tầng phải giữ đồng bộ giờ tăng từ một lên sáu.

1.5 Ngữ nghĩa mở và ngôn ngữ (Open-Vocabulary Semantics and Language)

Nhóm cuối cùng là nhóm ồn ào nhất, và với riêng bạn thì là nhóm ít liên quan nhất. Vì vậy mục này ngắn, và nó tập trung vào một câu hỏi duy nhất: *cái này giải bài toán của ai?*

A. Bốn công trình, một ý tưởng

Ý tưởng chung của cả nhóm là gán một véc-tơ đặc trưng vào từng phần của bản đồ 3D, sao cho véc-tơ đó nằm trong cùng không gian với đặc trưng của chữ. Khi đó truy vấn bằng chữ trở thành phép so cosin, và bạn không cần biết trước danh sách lớp.

- **ConceptFusion** hợp nhất đặc trưng đa phương thức mở vào bản đồ 3D, không cần huấn luyện thêm, và truy vấn được bằng chữ, bằng ảnh hoặc bằng một cú bấm [29]. Đóng góp kỹ thuật đáng chú ý là cách trộn đặc trưng cục bộ của từng vùng với đặc trưng toàn cục của cả ảnh, vì đặc trưng cục bộ đơn thuần mất ngữ cảnh.
- **LERF** nhúng đặc trưng ngôn ngữ vào một trường bức xạ ở nhiều mức tỉ lệ, rồi dựng ảnh chúng như dựng ảnh màu [30]. Kết quả là một trường “mức liên quan” trong không gian 3D.

Giới hạn nằm ngay trong cách làm: nó cho vùng chứ không cho ranh giới thực thể, nó cần tối ưu riêng cho từng cảnh, và nó chậm.

- **OpenScene** chứng cất đặc trưng mở từ ảnh 2D xuống một mạng 3D, nên mỗi điểm 3D mang sẵn một đặc trưng đồng không gian với chữ [31]. Ưu điểm so với LERF là không phải tối ưu lại cho từng cảnh.
- **SpatialLM** (2025) đi theo hướng khác: nhận đám mây điểm và sinh ra mô tả cảnh có cấu trúc — tường, cửa, cửa sổ, hộp bao vật thể — ở dạng máy đọc được. Tôi chưa kiểm chứng chi tiết công trình này, nên chỉ ghi nhận ý tưởng: dùng một mô hình ngôn ngữ làm bộ giải mã *bổ cục*, không phải làm bộ trả lời câu hỏi.

B. Cái này giải bài toán của ai

Trả lời thẳng: không phải của bạn, nếu đầu ra của hệ thống bạn là một quỹ đạo.

Không công trình nào trong nhóm này làm APE tốt lên. Chúng không chạm vào bốn nỗi đau của Phần F. Chúng cộng thêm một mô hình nền vào ngân sách vốn đã chật, và cộng thêm một chế độ hỏng mới — một truy vấn chữ trả về sai vật, không có cách nào chẩn đoán và không có ngưỡng nào để chỉnh. Với một hệ đo quỹ đạo, đây là chi phí thuần.

Nhưng câu trả lời đối hẩn khi người tiêu thụ bản đồ đổi. Nếu robot của bạn phải nhận lệnh “ra chỗ máy in lấy tập tài liệu”, thì không có tầng nào trong hệ hiện tại của bạn hiểu được chữ “máy in”. Đám mây điểm không có từ vựng. Đây đúng là tầng còn thiếu, và không có cách nào vòng qua nó bằng hình học. Với robot có nhiệm vụ, nhóm này chuyển từ “chi phí thuần” thành “điều kiện cần”.

Có đúng một đường mà nhóm này có thể quay lại giúp phần hình học, và tôi nêu ra như một giả thuyết chưa được kiểm chứng chứ không phải một kết luận. Đó là kiểm chứng vòng: một chòm sao vật thể có nhãn là bất biến với ánh sáng và với góc nhìn theo cách mà bộ mô tả vẽ ngoài không có. Hydra đã đi được một đoạn theo hướng này bằng cấu trúc phân tầng [22]. Liệu ngữ nghĩa mở có làm tốt hơn ngữ nghĩa tập đóng ở đây không thì tôi chưa thấy bằng chứng nào, và đó là một ablation đáng làm.

Bảng 1.2. Bốn nhóm trong nửa sau chương, đọc theo người tiêu thụ. Cột “ảnh hưởng tới APE” là cột phân loại nhanh nhất: nếu nó bằng không thì công trình đó thuộc về một cuộc trò chuyện khác, dù nó xuất hiện trong cùng một hội nghị và cùng một phiên.

Nhóm	Người tiêu thụ thật	Ảnh hưởng tới APE	Điều kiện để nó đáng làm
Lọc vật động (cột 1 + 2)	Bộ tối ưu tư thế	Có, cả hai chiều	Vật động vượt vùng RANSAC xử lý được; oracle cho thấy trần cải thiện đủ lớn
Bản đồ theo thời gian (cột 3)	Bản đồ sống qua nhiều phiên	Gián tiếp, qua chất lượng định vị lại	Robot đi lại nhiều lần cùng một chỗ, và cảnh thật sự đổi giữa các lần
Đồ thị cảnh phân tầng	Bộ lập kế hoạch; và tầng lấy ứng viên vòng	Gián tiếp, chỉ qua đóng vòng	Đã có sẵn bản đồ đặc; nếu chưa có thì đây là dự án nhiều tháng
Ngữ nghĩa mở, ngôn ngữ	Con người ra lệnh	Bằng không	Nhiệm vụ được phát biểu bằng chữ; không có nhiệm vụ như vậy thì đây là chi phí thuần

I 1.6 Giới hạn và trường hợp hỏng

Giới hạn nặng nhất đã nêu ở giữa chương, nhưng nó đáng được nhắc lại như một kết luận độc lập: **tầng ngữ nghĩa hỏng tương quan với tầng hình học, chứ không độc lập**. Ảnh mờ, thiếu sáng, chuyển động nhanh, phơi sáng nhảy — đó vừa là những khung hình mà ghép cặp đặc

trung yếu nhất, vừa là những khung hình nằm xa nhất khỏi phân phối huấn luyện của mạng. Một tầng dự phòng chỉ đáng tiền khi nó hỏng độc lập với tầng chính. Ở đây thì không, nên đừng thiết kế hệ thống dựa trên giả định rằng ngữ nghĩa sẽ đỡ cho bạn ở các khung hình khó.

Chế độ hỏng thứ hai là giả định khối cứng. Các hệ theo vết vật thể đều mô hình mỗi vật như một khối cứng có một tư thế. Con người thì không phải khối cứng, và con người lại chính là vật động phổ biến nhất trong mọi tập dữ liệu trong nhà. Hệ quả là các phương pháp đẹp nhất về mặt lý thuyết trong dòng này lại yếu nhất đúng ở trường hợp thường gặp nhất.

Chế độ hỏng thứ ba nằm ở tập nhãn. Một mô hình huấn luyện trên COCO biết tám mươi lớp, và nhà kho của bạn có xe nâng, pallet, tấm chắn, xe đẩy. Tập nhãn mở giải được về nguyên tắc, nhưng chi phí của nó đặt nó ra ngoài vòng lặp, nên trong thực tế bạn quay về đường chưng cất: mô hình lớn gán nhãn ngoại tuyến, mô hình nhỏ chạy trực tuyến. Đường đó có một điểm gãy riêng — mô hình nhỏ kế thừa luôn mọi sai lệch của mô hình lớn, và bạn không còn chân trị nào để phát hiện.

Chế độ hỏng thứ tư là chế độ hỏng của *đánh giá*, và nó âm thầm nhất. Các bảng kết quả trong mảng này phần lớn đo trên chuỗi trong nhà, RGB-D, người chiếm phần rất lớn khung hình, đối chứng là hệ không xử lý gì. Con số cải thiện lớn ở đó không chuyển sang chế độ vận hành của bạn. Muốn biết nó có chuyển hay không, chỉ có một cách: chạy thí nghiệm oracle trên dữ liệu của chính bạn, trước khi viết dòng code nào.

Riêng với đồ thị cảnh, ba giới hạn thêm. Khái niệm “phòng” không tồn tại ngoài trời và rất mơ hồ trong không gian mở, nên tầng phòng mất giá trị đúng ở môi trường mà bạn hay chạy nhất. Chi phí bản đồ đặc là ràng buộc chặn thật trên Orin, không phải chi phí mạng. Và việc giữ sáu tầng nhất quán sau mỗi lần tối ưu toàn cục là một nguồn bế tắc và trạng thái hỏng mà bạn đã nếm ở quy mô nhỏ hơn nhiều.

Cuối cùng, một giới hạn về chính chương này. Phần lớn công trình tôi nhắc sau năm 2023 tôi chỉ đọc chứ chưa chạy. Vì thế chương này cố ý *không* nêu một con số hiệu năng nào: không mili-giây, không FPS, không tỉ lệ cải thiện. Vài phép tính bằng tay còn lại đều có dẫn “giả sử” và chỉ nói về hình dạng của một quan hệ hình học, không nói về hiệu năng của hệ nào. Chỗ nào trước đây cần một con số thì bây giờ là một thứ hạng định tính kèm phép đo bạn phải tự chạy. Thứ hạng dùng để loại phương án; chỉ con số bạn tự đo trên máy mình mới dùng được để lập kế hoạch.

1.7 Thực hành

Repo và tài nguyên. Bắt đầu từ phía nhận thức: [ultralytics/ultralytics](#) cho dòng YOLO và cho đường xuất sang TensorRT, [open-mmlab/mmdetection](#) khi cần so nhiều họ detector trong cùng một khung, [facebookresearch/segment-anything](#) và [facebookresearch/sam2](#) để thử phân đoạn theo prompt, [IDEA-Research/GroundingDINO](#) khi cần một bộ gán nhãn tập mở chạy ngoại tuyến. Phía SLAM động: [BertaBescos/DynaSLAM](#) là bản tham chiếu ngắn gọn nhất để đọc, vì nó cắm thẳng vào ORB-SLAM2 nên bạn thấy rõ đúng chỗ mặt nạ can thiệp. Phía đồ thị cảnh: [MIT-SPARK/Kimera-VIO](#), [MIT-SPARK/Hydra](#) và [MIT-SPARK/Khronos](#) là ba bậc thang của cùng một dòng, đọc theo thứ tự đó thì thấy rõ mỗi bậc thêm gì. Phía tập nhãn mở: [concept-graphs/concept-graphs](#). Cuối cùng vẫn là [evo](#) để mọi con số bạn báo ra so được với nhau. Danh sách này chốt tại tháng 9/2026; tên mô hình dẫn đầu sẽ cũ trong sáu tới mười hai tháng, còn hạ tầng như TensorRT, [evo](#) và các thư viện dựng lưới thì bền hơn nhiều.

Câu hỏi kiểm tra hiểu. Trên chuỗi khó nhất của bạn, vật động chiếm bao nhiêu phần khung hình ở phân vị 95, và bạn đo con số đó bằng cách nào? Nếu mặt nạ của bạn xoá một phần lớn số đặc trưng, chính sách chèn khung hình chính có đổi không, và bạn kiểm điều đó ra sao? Một vật đi thẳng ra xa camera vi phạm ràng buộc hình học nào — và nếu câu trả lời là “không

cái nào”, thì cái gì phát hiện được nó? Đồ thị cảnh của Hydra cần thứ gì mà bản đồ điểm thưa của bạn không có, và thêm thứ đó tốn bao nhiêu bộ nhớ trên Orin?

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
● ● ● Mặt nạ ngữ nghĩa	Ảnh nhị phân đánh dấu các điểm ảnh thuộc lớp có thể di chuyển. Nó là tri thức tiên nghiệm về <i>lớp</i> , không phải phép đo về thực thể trước mặt, nên một mình nó không kết luận được vật đang động.
● ● ● Ba việc khác nhau khi loại vật động	Vật <i>có thể</i> động (ngữ nghĩa nói), vật <i>đang</i> động (hình học nói), vật <i>đã</i> động rồi đứng yên (chỉ trí nhớ của bản đồ nói). Không việc nào suy ra được việc nào, và mỗi việc có một kiểu hỏng riêng.
● ● Vòng luẩn quẩn tư thế–vật động	Muốn tính phần dư để phát hiện vật động thì cần tư thế camera, mà tư thế lại đang ước lượng từ chính tập điểm chứa vật động. Khi vật động chiếm phần lớn khung hình, kết luận đảo ngược hoàn toàn.
● ● ● Điều kiện của bài toán	Mức độ trải đủ hướng của tập ràng buộc còn lại. Xoá một mảng liên khối làm điểm dồn về một phía và tư thế có một hướng gần như không bị ràng buộc; không cờ ngoại lai nào bật lên, nên đây là kiểu hỏng im lặng.
● ● ● Hạ trọng số thay vì xoá	Nhân độ tin cậy của mặt nạ vào ma trận thông tin của phép đo, thay cho một quyết định nhị phân đặt ở đầu đường ống. Điểm nghi ngờ vẫn góp vào điều kiện của bài toán, và hàm mất mát bền vững vẫn đập được nó khi có bằng chứng thật.
● ● ● Thí nghiệm oracle cho mặt nạ	Dựng mặt nạ hoàn hảo bằng tay hoặc bằng mô hình nặng chạy ngoại tuyến, rồi đo cải thiện. Nó chặn trên mọi công sức làm mặt nạ thời gian thực; nếu trần đó nằm dưới sàn nhiều thì dừng ngay.
● ● ● DynaSLAM	Ghép mặt nạ phân đoạn thực thể cho các lớp có thể động với một kiểm tra sai lệch độ sâu đa khung, rồi vá lại nền. Bản tham chiếu gọn nhất của việc dùng cả hai nguồn bằng chứng; giá phải trả là một lượt mạng nặng mỗi khung. Họ hàng gần: DS-SLAM chạy mạng trên luồng riêng, DynaSLAM II đưa tư thế vật vào hiệu chỉnh chùm tia.
● ● ● SAM	Phân đoạn theo prompt: điểm, khung hoặc mặt nạ thô. Bộ mã hoá ảnh nặng chạy một lần mỗi ảnh, bộ giải mã nhẹ chạy một lần mỗi prompt. Nó <i>không</i> trả về nhãn lớp, nên một mình nó không lọc được vật động.
● ● Grounding DINO	Phát hiện vật theo mô tả bằng chữ, nên gỡ được ràng buộc tập nhãn đồng. Quá nặng cho vòng lặp trên Orin; vai trò đúng là bộ gán nhãn ngoại tuyến để chưng cất xuống một detector nhỏ. Open-YOLO 3D làm cùng tinh thần tiết kiệm ở tầng 3D bằng cách thay mặt nạ nặng bằng hộp.
● ● ● Đồ thị cảnh 3D	Bản đồ nhiều tầng: hình học, lưới có nhãn, vật thể và tác nhân, nơi chốn, phòng, toà nhà. Tốc độ thay đổi giảm dần khi lên cao, nên tầng trên dùng được làm mỏ neo. Kimera là thư viện mở dựng tầng lưới có nhãn, với các mô-đun thay được từng cái.
● ● Hydra	Dựng đồ thị cảnh tăng dần theo thời gian thực. Việc khó nhất không phải nhận dạng mà là biến dạng cả sáu tầng cho nhất quán mỗi khi một vòng đóng lại. Nó cũng phát hiện vòng bằng cấu trúc phân tầng thay vì bằng vẽ ngoài, nên có chế độ hỏng khác.
● ● Clio	Mức chi tiết của bản đồ được quyết định bởi <i>danh sách nhiệm vụ</i> , qua nút thắt thông tin. Một chồng sách là một vật hay mười vật tùy nhiệm vụ; “vật thể” không phải thuộc tính của cảnh.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Khronos	Bỏ phân loại tĩnh–động, thay bằng ước lượng khoảng thời gian tồn tại của từng vật. Nhờ đó chuyển động ngắn hạn và thay đổi dài hạn dùng chung một cơ chế, và bản đồ trả lời được “lúc t thế giới trông thế nào”.
●● Ngữ nghĩa mở	ConceptFusion, LERF và OpenScene gắn vào bản đồ 3D các đặc trưng nằm cùng không gian với chữ, nên truy vấn được bằng ngôn ngữ; ConceptGraphs ghép chúng thành đồ thị. Không cái nào làm APE tốt lên: chúng phục vụ người ra lệnh, không phục vụ bộ ước lượng tư thế.
●●● Vì sao RANSAC không đủ?	RANSAC hổng khi ngoại lai đông và nhất quán, vì khi đó chúng thắng phiếu đồng thuận. Vật động tạo sai lệch <i>có hướng</i> chứ không phải nhiễu ngẫu nhiên, nên cả cụm cùng bỏ phiếu cho một nghiệm sai. Đó là vùng duy nhất mặt nạ thật sự đáng tiền.
●●● Vì sao mặt nạ ngữ nghĩa một mình không đủ?	Sáu lý do: nhân nói về lớp chứ không về thực thể; có vật động ngoài tập nhân; có chuyển động gián tiếp như bóng đổ và phản chiếu; biên mặt nạ lệch đúng chỗ góc ORB nằm; mặt nạ luôn cũ vài khung; và mạng hổng đúng ở khung hình khó.
●● Vì sao bỏ triết phi cực đại lại quan trọng với hệ thời gian thực?	Vì thời gian hậu xử lý tỉ lệ với số khung sống sót, nên đuôi độ trễ dày lên đúng ở cảnh đông người. YOLOv10 bỏ nó bằng hai phép gán nhất quán, RT-DETR bỏ nó bằng dự đoán tập hợp; cái được là độ trễ hằng số, không phải mAP.
●●● Vì sao mạng hổng đúng lúc cần nó nhất?	Ảnh mờ, thiếu sáng và chuyển động nhanh vừa làm hình học yếu, vừa nằm ngoài phân phối huấn luyện của mạng. Hai kiểu hổng tương quan dương, nên tầng ngữ nghĩa không dùng được làm lưới an toàn.
●● APE đẹp lên nhưng chuỗi hay mất bám — nghĩa là gì?	Gần như chắc chắn là xoá quá tay: APE chỉ tính trên đoạn còn sống nên hệ mất bám sớm lại có APE đẹp. Luôn báo tỉ lệ hoàn thành cùng APE, và so cả hai với sản phẩm đo từ nhiều lần chạy.
●● Bật mặt nạ thì số khung hình chính giảm hẳn — nghĩa là gì?	Mặt nạ đã đổi luôn chính sách chèn khung hình chính, nên bạn đang so hai đồ thị khác nhau chứ không phải hai bộ phép đo. Cách phân biệt: khoá chính sách khung hình chính lại rồi chạy lại, hoặc báo cáo số khung hình chính kèm APE.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
BertaBescos/DynaSLAM [14]	Ghép cột 1 với cột 2: mặt nạ ngữ nghĩa cho “có thể động”, kiểm tra độ sâu đa khung cho “đang động”	Bản tham chiếu ngắn nhất để đọc; cắm thẳng vào ORB-SLAM2
MIT-SPARK/Hydra [22]	Dựng đồ thị cảnh phân tầng theo thời gian thực và biến dạng cả sáu tầng khi vòng đóng	Đang được bảo trì; là nền cho Clio và Khronos
Khronos [18]	Thay phân loại tĩnh–động bằng khoảng thời gian tồn tại của từng vật — lời giải cho cột 3	Cần bản đồ đặc và cần nhiều lần đi qua
Clio [26]	Mức chi tiết của bản đồ suy ra từ danh sách nhiệm vụ, qua nút thất thông tin	Bài học thiết kế rộng hơn phạm vi đồ thị cảnh
ultralytics	Dòng detector nhẹ chạy được trong vòng lặp; đường xuất sang TensorRT là phần dùng nhiều nhất	Hạ tầng; bền hơn tên phiên bản mô hình
Dòng SAM [10], [11]	Cấu trúc chi phí “mã hoá một lần, giải mã theo prompt” — mẫu đặt phần đất ở chỗ được khẩu hao	Chủ yếu dùng ngoại tuyến để gán nhãn dữ liệu

MINI PROJECT SLAM · Đo trần của việc lọc vật động trên dữ liệu của bạn · 1–2 ngày

Mục tiêu	trả lời câu hỏi “có đáng làm không” bằng một con số, <i>trước</i> khi bỏ công làm mặt nạ chạy thời gian thực. Đây là thí nghiệm oracle của chương 24, áp cho tầng ngữ nghĩa.
Dữ liệu	một chuỗi có người đi lại thật — OpenLORIS, hoặc một chuỗi bạn tự quay ở hành lang có người qua lại. Kèm một chuỗi EuRoC làm đối chứng âm, vì EuRoC gần như không có vật động và kết quả ở đó phải là “không đối gì”.
Các bước	trước hết đo phân bố diện tích vật động: chạy một mô hình phân đoạn nặng ngoại tuyến, ghi tỉ lệ điểm ảnh mang nhãn động cho từng khung, lấy phân vị 95. Sau đó dựng <i>mặt nạ oracle</i> bằng chính mô hình nặng đó chạy ngoại tuyến, nở biên vài điểm ảnh, lưu ra đĩa. Chạy ba cấu hình, mỗi cấu hình mười lần: không mặt nạ, mặt nạ oracle dạng xoá cứng, mặt nạ oracle dạng hạ trọng số. Với mỗi lần chạy ghi ba số chứ không phải một: APE, tỉ lệ hoàn thành, và số khung hình chính.
Xong khi	bạn có một bảng ba cấu hình nhân ba số, kèm độ lệch chuẩn qua mười lần chạy, và bạn phát biểu được một trong hai câu: “mặt nạ hoàn hảo cải thiện APE thêm X cm, vượt sàn nhiều, nên bản thời gian thực đáng làm”, hoặc “mặt nạ hoàn hảo không vượt sàn nhiều trên chuỗi này, nên dừng”. Kiểm tra chéo bắt buộc: trên EuRoC cả ba cấu hình phải trùng nhau trong sàn nhiều; nếu không, đường ống mặt nạ của bạn đang đổi một thứ khác ngoài mặt nạ.
Nối với	E/24 cho thí nghiệm oracle và cỡ mẫu; B/07 cho cách đọc cặp APE và tỉ lệ hoàn thành; kết quả của dự án này là điều kiện vào của dự án thứ hai bên dưới.

MINI PROJECT SLAM · Ngân sách thật của mạng nhận thức trên Orin · 1 ngày

Mục tiêu	biến con số FPS trong paper thành con số dùng được, bằng cách đo trong đúng điều kiện mà nó sẽ chạy: cùng máy, cùng lúc, cùng tranh chấp tài nguyên.
Dữ liệu	một chuỗi bất kỳ phát lại ở tốc độ thật trên Jetson Orin, cộng một detector nhẹ đã xuất sang TensorRT ở nửa độ chính xác.
Các bước	đo ba lần. Lần một: chỉ chạy mạng, ghi trung bình và p99 của độ trễ. Lần hai: chỉ chạy SLAM, ghi trung bình và p99 của thời gian mỗi khung, cùng tỉ lệ khung bị bỏ. Lần ba: chạy cả hai cùng lúc, ghi lại đúng bốn con số đó. Sau đó lặp lần ba với mạng đặt ở tần số một phần ba nhịp khung, chỉ trên khung hình chính.
Xong khi	bạn có một bảng bốn dòng cho thấy p99 của cả hai bên tăng bao nhiêu khi chạy chung, và bạn nói được tần số cao nhất mà mạng chạy được mà không đẩy p99 của luồng theo vết vượt ngân sách khung. Nếu p99 chạy chung gần bằng tổng hai p99 riêng, bạn vừa xác nhận rằng “song song” ở đây là chạy xen kẽ.
Nối với	D/21-inference-va-trien-khai cho phần xuất mô hình và lượng tử hoá; B/11 cho cách đọc con số theo mô hình roofigline; F/34 cho phần đo lại trung thực trên phần cứng thật.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì bộ tối ưu không cần nhiều ràng buộc mà cần ràng buộc trái đủ hướng. Mặt nạ luôn liên khối, nên xoá nó làm các điểm còn lại dồn về một phía và tư thế có một hướng gần như không bị ràng buộc. Kiểu hỏng này im lặng: phần dư còn lại nhỏ và đẹp, hiệp phương sai báo ra thì lạc quan, và sai số chỉ hiện ra vài giây sau dưới dạng trôi.
- A2. Đúng một loại: ngoại lai *đồng và nhất quán*. RANSAC chọn nghiệm được nhiều phiếu nhất, nên khi vật động chiếm phần lớn khung hình và cùng chuyển động theo một hướng, chính chúng thắng phiếu và nền tĩnh bị coi là ngoại lai. Ngữ nghĩa phá thế bế tắc bằng một lá phiếu đến từ ngoài hình học. Ngoài vùng đó, nó chủ yếu lấy đi dữ liệu tốt.
- A3. Không trùng hợp, và đó là kết luận thiết kế quan trọng nhất của chương. Ảnh mờ, thiếu sáng và chuyển động nhanh vừa là các khung hình mà ghép cặp yếu nhất, vừa là các khung hình xa nhất khỏi phân phối huấn luyện của mạng. Một tầng dự phòng chỉ đáng tiền khi nó hỏng độc lập với tầng chính; ở đây hai kiểu hỏng tương quan dương.

- A4. Cái thứ nhất trả lời “lớp này có khả năng tự di chuyển không” và trả lời được ngay từ khung hình đầu, nhưng nó nói về lớp chứ không về thực thể. Cái thứ hai trả lời “vật này đang động ở khung hình này không” nhưng cần một tư thế đủ tốt, tức là mắc vòng luẩn quẩn. Cái thứ ba trả lời “chỗ này đã đổi so với lần trước chưa” và chỉ trả lời được sau nhiều lần đi qua; nó là bài toán bảo trì bản đồ, không phải bài toán nhận thức.
- A5. Nó không giúp gì cả, và câu trả lời trung thực đó chính là công cụ phân loại. Nếu người tiêu thụ bản đồ là một script tính APE, tầng ngôn ngữ là chi phí thuần cộng thêm một chế độ hồng không chẩn đoán được. Nếu người tiêu thụ là một bộ lập kế hoạch nhận lệnh bằng chữ, thì đám mây điểm không có từ vựng và đây đúng là tầng còn thiếu. Hãy hỏi “ai đọc bản đồ này” trước khi hỏi “phương pháp nào tốt hơn”.

NÓI LẠI THẬT ĐƠN GIẢN

Tưởng tượng bạn vẽ lại sơ đồ một siêu thị bằng cách đi vòng quanh và ghi nhớ đồ vật ở đâu. Cách này chạy tốt, cho tới khi bạn ghi nhớ nhầm một hộp ngũ cốc nằm trong xe đẩy của người khác. Hôm sau bạn quay lại, hộp ngũ cốc đã đi mất. Nhưng bạn thì tin nó vẫn ở chỗ cũ, nên bạn kết luận rằng *chính bạn* mới là người đang đứng sai chỗ. Toàn bộ sơ đồ trượt theo.

Mẹo là dạy cho máy biết loại đồ nào hay nằm trong xe đẩy, rồi bảo nó bớt tin những thứ đó. Mẹo này ăn tiền vì máy học được từ hàng triệu bức ảnh, còn bản thân phép đo thì không thể tự biết điều đó.

Cái giá thì có ba phần, và cả ba đều thật. Thứ nhất, máy chỉ biết *loại* đồ, không biết cái hộp trước mặt nó lúc này đang nằm trên kệ hay trong xe. Thứ hai, nếu nó gạch bỏ mọi thứ có thể nằm trong xe đẩy, nó có thể gạch nguyên một dãy kệ, và rồi cả một phía của sơ đồ không còn mốc nào để dựa. Thứ ba, nhìn ra đồ vật tốn thời gian, mà mỗi bước chân bạn chỉ có một khoảnh khắc rất ngắn.

Chỗ không rút gọn được: vì sao gạch bỏ dữ liệu *đúng* lại làm sơ đồ tệ đi. Mọi cách nói gọn đều thành “cần mốc rải đều”, nghe như một sự tiện lợi. Sự thật là một phát biểu về hình dạng của một ma trận, và tôi chưa tìm được cách nói ngắn nào vừa đúng vừa không cần tới ma trận.

Một câu: *Máy chỉ biết loại đồ nào hay bị xô dịch, chứ không biết cái trước mặt nó lúc này có xô dịch hay không — nên đừng bảo nó xoá, hãy bảo nó bớt tin.*

1.8 Kết nối

- **C/14-bai-toan-cv-loi — tiền đề.** Chương 14 dạy detection và segmentation là gì: gán mẫu, triệt phi cực đại, dự đoán tập hợp, phân loại mặt nạ. Chương này chỉ dùng chúng, và chỗ nổi là câu hỏi “mô hình nào bỏ vừa vào ngân sách khung hình”.
- **F/28-dac-trung-va-ghep-cap — cùng nguyên lý với.** Mặt nạ ngữ nghĩa và bộ mô tả học được can thiệp vào đúng cùng một chỗ: quyết định của frontend về việc tin phép đo nào. Cái thứ nhất đổi *trọng số*, cái thứ hai đổi *chất lượng*. Hai đòn bẫy độc lập, và nên được đo tách rời nhau.
- **F/29-dinh-vi-lai-va-dong-vong — hệ quả của.** Vòng sai vì hai chỗ chia nhau cùng một chiếc xe đẩy là một chế độ hồng của tầng lấy ứng viên. Đồ thị cảnh phân tầng đề xuất một tầng lấy ứng viên khác, có chế độ hồng khác — và “chế độ hồng khác” mới là thứ đáng giá khi ghép hai bộ phát hiện lại.
- **F/30-do-sau-dong-quang-va-mo-hinh-nen — tiền đề.** Cột 2 của Hình 1.3 chạy bằng dòng quang và độ sâu. Chất lượng phát hiện “đang động” bị chặn trên bởi chất lượng của hai thứ đó, nên hai chương phải đọc cùng nhau.
- **F/32-ban-do-neural — đối lập với.** Cả hai chương đều thay bản đồ điểm thừa bằng thứ khác. Bản đồ neural đi về phía *đặc và liên tục*, tối ưu cho việc dựng lại ảnh. Đồ thị cảnh đi về phía *thưa và có ký hiệu*, tối ưu cho việc suy luận. Hai hướng ngược nhau, và chúng phục vụ hai người tiêu thụ khác nhau.

- **F/34-imu-bat-dinh-va-trien-khai** — cùng nguyên lý với. Kết luận “hạ trọng số thay vì xoá” là một trường hợp riêng của luận điểm chính chương 34: mọi quyết định tin cậy trong đồ thị nhân tử cuối cùng đều là một con số trong ma trận thông tin. Chương 34 cũng cho một nguồn bằng chứng mà chương này không dùng tới: IMU biết camera đã chuyển động thế nào, nên nó giúp tách chuyển động của vật khỏi chuyển động của người quan sát.
- **B/08-dich-chuyen-phan-phoi** — trường hợp riêng của. Việc mạng hồng đúng ở khung hình khó là dịch chuyển phân phối ở dạng độc nhất, vì hướng dịch chuyển trùng với hướng làm hình học yếu đi.
- **E/24-thi-nghiem-va-ablation** — tiền đề. Thí nghiệm oracle và quy tắc đo sản nhiều trước đều lấy nguyên từ chương 24. Không có hai công cụ đó, mọi kết luận trong chương này đều không kiểm chứng được.
- **B/11-gioi-han-tinh-toan** — ràng buộc bởi. Bảng 1.1 chỉ là chương 11 áp cho một loại phần cứng cụ thể; điều đáng mang đi là thói quen bắt đầu từ ngân sách chứ không từ bảng độ chính xác.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao một vật động tạo ra sai lệch *có hướng* chứ không phải nhiễu ngẫu nhiên, và điều đó thay đổi cách bạn chọn hàm mất mát bền vững ra sao?
2. Nếu mặt nạ của bạn nở thêm vài điểm ảnh ở biên, bạn được gì và mất gì — và bạn đo hai thứ đó bằng phép đo nào?
3. Một chuỗi cho APE tốt lên rõ rệt khi bật mặt nạ, nhưng số khung hình chính cũng giảm hẳn. Bạn kết luận được gì, và thí nghiệm nào tách được hai hiệu ứng đó?
4. Vì sao thí nghiệm oracle phải chạy *trước* khi bạn viết dòng code nào cho mặt nạ thời gian thực, và điều gì xảy ra nếu bạn đảo thứ tự?
5. Đồ thị cảnh phân tầng phát hiện vòng bằng cấu trúc thay vì bằng vẻ ngoài. Chế độ hồng của nó khác chế độ hồng của bộ mô tả ảnh toàn cục ở đâu, và vì sao sự khác biệt đó mới là lý do đáng ghép hai bộ lại?
6. Nếu bạn phải chọn giữa một detector nhanh hơn hai lần và một detector có độ trễ ổn định hơn hai lần, bạn chọn cái nào cho vòng lặp theo vết, và lý do nằm ở đại lượng nào?
7. Vì sao “bản đồ theo thời gian” không phải là một bài toán nhận thức, và việc ném thêm một mạng phân đoạn vào nó hồng ở bước nào?

Nguồn

- [1] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [2] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [3] J. Redmon, S. Divvala, R. Girshick, et al., “You Only Look Once: Unified, Real-Time Object Detection,” in *CVPR*, 2016.
- [4] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” *arXiv preprint arXiv:2004.10934*, 2020.
- [5] Z. Ge, S. Liu, F. Wang, et al., “YOLOX: Exceeding YOLO Series in 2021,” *arXiv preprint arXiv:2107.08430*, 2021.
- [6] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, “YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [7] A. Wang, H. Chen, L. Liu, et al., “YOLOv10: Real-Time End-to-End Object Detection,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [8] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [9] Y. Zhao, W. Lv, S. Xu, et al., “DETRs Beat YOLOs on Real-time Object Detection,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [10] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [11] N. Ravi, V. Gabeur, Y.-T. Hu, et al., “SAM 2: Segment Anything in Images and Videos,” *arXiv preprint arXiv:2408.00714*, 2024.
- [12] S. Liu, Z. Zeng, T. Ren, et al., “Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [13] M. E. A. Boudjoghra, A. Dai, J. Lahoud, et al., “Open-YOLO 3D: Towards Fast and Accurate Open-Vocabulary 3D Instance Segmentation,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [14] B. Bescos, J. M. Fácil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018.
- [15] C. Yu, Z. Liu, X.-J. Liu, et al., “DS-SLAM: A Semantic Visual SLAM towards Dynamic Environments,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [16] M. Rünz, M. Buffier, and L. Agapito, “MaskFusion: Real-Time Recognition, Tracking and Reconstruction of Multiple Moving Objects,” in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2018.
- [17] B. Bescos, C. Campos, J. D. Tardós, et al., “DynaSLAM II: Tightly-Coupled Multi-Object Tracking and SLAM,” *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5191–5198, 2021.

- [18] L. Schmid, M. Abate, Y. Chang, et al., “Khronos: A Unified Approach for Spatio-Temporal Metric-Semantic SLAM in Dynamic Environments,” in *Robotics: Science and Systems (RSS)*, 2024.
- [19] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [20] J. Bergstra and Y. Bengio, “Random Search for Hyper-Parameter Optimization,” in *JMLR*, 2012.
- [21] A. Rosinol, A. Gupta, M. Abate, et al., “3D Dynamic Scene Graphs: Actionable Spatial Perception with Places, Objects, and Humans,” in *Robotics: Science and Systems (RSS)*, 2020.
- [22] N. Hughes, Y. Chang, and L. Carlone, “Hydra: A Real-time Spatial Perception System for 3D Scene Graph Construction and Optimization,” in *Robotics: Science and Systems (RSS)*, 2022.
- [23] J. McCormac, A. Handa, A. Davison, et al., “SemanticFusion: Dense 3D Semantic Mapping with Convolutional Neural Networks,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [24] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [25] Y. Chang, N. Hughes, A. Ray, et al., “Hydra-Multi: Collaborative Online Construction of 3D Scene Graphs with Multi-Robot Teams,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023.
- [26] D. Maggio, Y. Chang, N. Hughes, et al., “Clio: Real-time Task-Driven Open-Set 3D Scene Graphs,” *IEEE Robotics and Automation Letters*, 2024.
- [27] Q. Gu, A. Kuwajerwala, S. Morin, et al., “ConceptGraphs: Open-Vocabulary 3D Scene Graphs for Perception and Planning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [28] J. Straub, D. DeTone, T. Shen, et al., “EFM3D: A Benchmark for Measuring Progress Towards 3D Egocentric Foundation Models,” *arXiv preprint arXiv:2406.10224*, 2024.
- [29] K. M. Jatavallabhula, A. Kuwajerwala, Q. Gu, et al., “ConceptFusion: Open-set Multimodal 3D Mapping,” in *Robotics: Science and Systems (RSS)*, 2023.
- [30] J. Kerr, C. M. Kim, K. Goldberg, et al., “LERF: Language Embedded Radiance Fields,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [31] S. Peng, K. Genova, C. Jiang, et al., “OpenScene: 3D Scene Understanding with Open Vocabularies,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.